

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



HỌ TÊN SINH VIÊN: PHẠM QUỐC KHÁNH
 TRẦN TIẾN HỢP
 HUỲNH THIÊN BẢO
 LÊ NHỰT TRƯỜNG
 TRẦN HUỲNH HÒA PHÚC

ĐỀ TÀI: THIẾT KẾ TRANG WEB ĐẶT TOUR DU LỊCH

BÁO CÁO ĐỒ ÁN MÔN HỌC

Tp. Hồ Chí Minh – Năm 2025

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN



HỌ TÊN SINH VIÊN: PHẠM QUỐC KHÁNH
TRẦN TIẾN HỢP
HUỲNH THIÊN BẢO
LÊ NHỰT TRƯỜNG
TRẦN HUỲNH HÒA PHÚC

ĐỀ TÀI: THIẾT KẾ TRANG WEB ĐẶT TOUR DU LỊCH

Lớp: CQ.65.CNTT
Khóa: 65

NGƯỜI HƯỚNG DẪN KHOA HỌC:

1. HDC: ThS. NGUYỄN THIÊN DƯƠNG

Tp. Hồ Chí Minh – Năm 2025

LỜI CẢM ƠN

Lời nói đầu tiên, em xin gửi tới Quý Thầy Cô Bộ môn Công nghệ Thông tin Trường Đại học Giao thông vận tải phân hiệu tại thành phố Hồ Chí Minh lời chúc sức khỏe và lòng biết ơn sâu sắc.

Em xin chân thành cảm ơn quý thầy cô đã giúp đỡ tạo điều kiện để em hoàn thành báo cáo bài tập lớn đề tài Thiết kế và xây dựng website du lịch Travel Explorer. Đặc biệt em xin cảm ơn thầy Nguyễn Thiện Dương đã nhiệt tình giúp đỡ, hướng dẫn cho em kiến thức về thiết kế web, lập trình front-end với HTML5, CSS3 và JavaScript, định hướng và kỹ năng để có thể hoàn thành bài báo cáo này.

Tuy đã cố gắng trong quá trình nghiên cứu tìm hiểu và phát triển website, tuy nhiên do kiến thức còn hạn chế nên vẫn còn tồn tại nhiều thiếu sót. Vì vậy em rất mong nhận được sự đóng góp ý kiến của Quý thầy cô bộ môn để đề tài của em có thể hoàn thiện hơn.

Lời sau cùng, em xin gửi lời chúc tới Quý Thầy Cô Bộ môn Công nghệ thông tin và hơn hết là thầy Nguyễn Thiện Dương có thật nhiều sức khỏe, có nhiều thành công trong công việc. Em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp. Hồ Chí Minh, ngày ... tháng ... năm ...

Giảng viên hướng dẫn

ThS. Nguyễn Thiện Dương

MỤC LỤC

LỜI CẢM ƠN	i
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	ii
MỤC LỤC	iii
BẢNG BIỂU, SƠ ĐỒ, HÌNH VẼ	vii
CHƯƠNG 1: MỞ ĐẦU	1
1.1. Giới thiệu đề tài.....	1
1.2. Mục tiêu đề tài.....	1
1.3. Mục đích nghiên cứu	2
1.4. Đối tượng nghiên cứu	2
1.5. Phạm vi nghiên cứu	2
1.6. Phương pháp nghiên cứu	3
1.7. Ý nghĩa thực tiễn của đề tài	3
1.8. Cấu trúc báo cáo	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ THỰC NGHIỆM	5
2.1. Cơ sở lý thuyết	5
2.1.1. HTML	5
2.1.1.1. Khái niệm	5
2.1.1.2. Các thành phần ngữ nghĩa	5
2.1.1.3. Khối HTML và các phần tử nội tuyến	7
2.1.1.4. HTML Table.....	8
2.1.1.5. HTML Form	8
2.1.1.6. Các thẻ hay sử dụng.	8
2.1.2. CSS	9
2.1.2.1. Bộ chọn (CSS Selectors).....	9
2.1.2.2. Các đơn vị đo (CSS Units)	11
2.1.2.3. Kích thước (Dimensions)	13
2.1.2.4. Mô hình hộp (CSS Box Model)	16
2.1.2.5. Đường viền (Borders).....	18
2.1.2.6. Thuộc tính hiển thị (Display Property)	19

2.1.2.7. Định vị phần tử (Positioning)	21
2.1.2.8. Nền (Background)	24
2.1.3. JavaScript	25
2.1.3.1. Khái niệm JavaScript.....	25
2.1.3.2. Các tính năng của JavaScript	25
2.1.3.3. Các mô hình lập trình trong JavsScript.....	26
2.1.3.4. Hạn chế của JavaScript	27
2.1.4. Lý thuyết màu.....	28
2.1.4.1. Hệ màu CMYK.....	28
2.1.4.2. Hệ màu RGB.....	28
2.1.4.3. Hệ màu HSB	28
2.1.4.4. Mã màu HEX	29
2.1.5. Bố cục và kiểu chữ.	29
2.1.5.1. Tạo kiểu văn bản (Styling text)	29
2.1.5.2. Bố cục (Flexbox)	29
2.1.5.3. Lưới bố cục (Grid Layout)	30
2.1.6. Các framework và thư viện phổ biến	30
2.1.6.1. Khái niệm Thư viện JavaScript	30
2.1.6.2. Một số Thư viện JavaScript Tiêu biểu	31
2.1.6.3. Khái niệm framework.....	32
2.1.6.4. Các framework phổ biến.	32
2.1.6.5. Sự khác nhau giữa framework và thư viện.....	33
2.1.7. Khái niệm về hosting, domain, sever.	34
2.1.7.1. Domain (Tên miền).....	34
2.1.7.2. Hosting (Web Hosting).....	35
2.1.7.3. Server (Máy chủ).....	35
2.2. Khảo sát các sản phẩm tương tự.....	36
2.2.1. Phân tích các website cùng lĩnh vực.....	36
2.2.2. Bảng so sánh các website	39
2.2.3. Nhận xét và định hướng cho sản phẩm của nhóm.....	40
2.3. Tổng hợp và định hướng thiết kế	40
2.3.1. Tóm tắt các yếu tố sẽ áp dụng cho dự án	40
2.3.2. Lý do chọn framework, phong cách thiết kế cụ thể	43

CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI WEBSITE	45
3.1. Thiết kế giao diện (UI Design)	45
3.1.1. Phân tích SiteMap	45
3.1.2. Bố cục tổng thể:	46
3.1.3. Wireframe hoặc mockup của các trang chính	47
3.1.3.1. Trang Chủ :	47
3.1.3.2. Trang Điểm Đến :	48
3.1.3.3. Trang Dịch Vụ :	48
3.1.3. Bộ màu, font chữ, hình ảnh, biểu tượng	48
3.1.4. Nguyên tắc UX/UI được áp dụng	51
3.1.5. Responsive trên desktop / tablet / mobile	52
3.2. Chức năng và Tương tác (UX Design)	53
3.2.1. Các chức năng chính của website	53
3.2.2. Phân tích sơ đồ USE CASE UML	54
3.2.2.1. Tác nhân (Actor)	55
3.2.2.2. Các Use Case trong hệ thống	55
3.2.3. Các đoạn mã và logic quan trọng	57
3.2.3.1. Logic Image Slider (Section 1 – Trang Điểm Đến)	57
3.2.3.2. Logic Mobile Menu Toggle	57
3.2.3.3. Logic Smooth Scrolling có bù trừ chiều cao Header	58
3.2.3.4. Logic Form Validation	58
3.3. Mã nguồn và cấu trúc dự án	58
3.3.1. Cấu trúc thư mục	58
3.3.2. Công cụ và thư viện sử dụng	59
3.3.3. Nguyên tắc đặt tên, tái sử dụng mã, comment code	59
CHƯƠNG 4: KIỂM THỬ, TỐI ƯU HÓA VÀ TRIỂN KHAI	62
4.1. Kiểm thử giao diện và chức năng	62
4.1.1. Kiểm tra từng chức năng	62
4.1.2. Kiểm thử responsive trên nhiều thiết bị	63
4.1.3. Kiểm tra tương thích trên các trình duyệt phổ biến	63
4.2. Tối ưu hóa hiệu suất	64
4.2.1. Tối ưu hình ảnh	64
4.2.2. Tối ưu CSS và JavaScript	64

4.2.3. SEO cơ bản	64
4.3. Triển khai website lên host và server	65
4.3.1. Giới thiệu hosting	65
4.3.2. Các bước triển khai và cấu hình.....	66
4.3.3. Link truy cập website	67
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	68
5.1. Kết quả đạt được	68
5.1.1. Những tiêu chí rubric đã hoàn thành.....	68
5.1.2. Mức độ đạt được so với mục tiêu ban đầu.....	70
5.1.3. Những đóng góp và thành tựu nổi bật	71
5.2. Hạn chế	71
5.3. Hướng phát triển	72
5.3.1. Đề xuất cải tiến về backend và cơ sở dữ liệu	72
5.3.2. Đề xuất cải tiến về chức năng người dùng.....	73
5.3.3. Đề xuất cải tiến về quản trị hệ thống.....	74
5.3.4. Đề xuất cải tiến về tính năng nâng cao.....	74
TÀI LIỆU THAM KHẢO.....	76
PHỤ LỤC	77

BẢNG BIỂU, SƠ ĐỒ, HÌNH VẼ

Hình 2.1. Cấu trúc trang HTML.....	5
Hình 2.2. Layout cơ bản web	7
Bảng 2.1. Đơn vị tuyệt đối trong CSS.....	11
Bảng 2.2. Đơn vị tương đối trong CSS	13
Hình 2.3. Mô hình hộp CSS	16
Bảng 2.3. Giá trị thông dụng của thuộc tính Display	20
Hình 2.4. Các đặc điểm chính của JavaScript	25
Hình 2.5. Ứng dụng của JavaScript.....	27
Hình 2.6. Sự khác biệt cơ bản giữa Framework và Library	34
Hình 2.7. Giao diện trang web booking.com	36
Hình 2.8. Giao diện trang web traveloka.com	37
Hình 2.9. Giao diện trang web viettravel.com	38
Bảng 2.4. So sánh các website	39
Hình 3.1. SiteMap chương trình.....	45
Hình 3.2. Layout trang chủ.....	47
Hình 3.3. Layout trang điểm đến.....	48
Hình 3.4. Layout trang dịch vụ.....	48
Hình 3.5. USE CASE UML	55
Hình 3.6. Cấu trúc thư mục	58

CHƯƠNG 1: MỞ ĐẦU

Chương 1 trình bày tổng quan về đề tài thiết kế và xây dựng website du lịch Travel Explorer. Chương này giới thiệu về đề tài, xác định mục tiêu, mục đích nghiên cứu, phạm vi và đối tượng nghiên cứu, phương pháp nghiên cứu, ý nghĩa thực tiễn và cấu trúc báo cáo đồ án. Đây là chương mở đầu, tạo nền tảng và định hướng cho toàn bộ quá trình nghiên cứu và phát triển website du lịch.

1.1. Giới thiệu đề tài

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ và nhu cầu du lịch ngày càng tăng cao, việc ứng dụng công nghệ web vào lĩnh vực du lịch đã trở thành xu hướng tất yếu. Website du lịch không chỉ là công cụ quảng bá điểm đến mà còn là nền tảng tích hợp đa dịch vụ, giúp khách hàng dễ dàng tìm kiếm, so sánh và đặt các dịch vụ du lịch một cách nhanh chóng và tiện lợi.

Đề tài "Thiết kế và xây dựng website du lịch Travel Explorer" được thực hiện nhằm phát triển một hệ thống website toàn diện, cung cấp các dịch vụ du lịch trực tuyến bao gồm: tìm kiếm và đặt tour du lịch, đặt phòng khách sạn, đặt vé máy bay, thuê xe du lịch, hỗ trợ làm visa và tư vấn du lịch. Website được thiết kế với giao diện hiện đại, thân thiện với người dùng, tối ưu hóa cho các thiết bị di động.

1.2. Mục tiêu đề tài

Mục tiêu chính của đề tài là thiết kế và xây dựng thành công một website du lịch hoàn chỉnh với các mục tiêu cụ thể:

- Mục tiêu kỹ thuật: Xây dựng website responsive, tương thích với nhiều trình duyệt và thiết bị, sử dụng HTML, CSS, JavaScript.
- Mục tiêu chức năng: Phát triển các module chính bao gồm quản lý điểm đến, tìm kiếm và đặt tour, đặt phòng khách sạn, đặt vé máy bay, thuê xe, hỗ trợ visa và tư vấn du lịch.
- Mục tiêu giao diện: Thiết kế giao diện đẹp mắt, hiện đại, dễ sử dụng.
- Mục tiêu trải nghiệm người dùng: Đảm bảo website có tốc độ tải nhanh, điều hướng dễ dàng, quy trình đặt dịch vụ đơn giản.

1.3. Mục đích nghiên cứu

Nghiên cứu này nhằm mục đích:

- Nghiên cứu và áp dụng các công nghệ web hiện đại (HTML5, CSS3, JavaScript) trong việc xây dựng website du lịch.
- Phân tích nhu cầu thực tế của người dùng để thiết kế các tính năng phù hợp.
- Xây dựng một hệ thống website hoàn chỉnh, có thể mở rộng và phát triển trong tương lai.
- Tạo ra sản phẩm thực tế có thể ứng dụng, giúp các công ty du lịch quảng bá dịch vụ và tăng hiệu quả kinh doanh.
- Rèn luyện kỹ năng phân tích, thiết kế và phát triển phần mềm web.

1.4. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm:

- Hệ thống website du lịch: Cấu trúc, chức năng, giao diện và trải nghiệm người dùng của website Travel Explorer.
- Công nghệ web: Các công nghệ HTML5, CSS3, JavaScript và cách ứng dụng trong xây dựng website du lịch.
- Quy trình nghiệp vụ du lịch: Các quy trình tìm kiếm tour, đặt phòng, đặt vé để số hóa và tích hợp vào website.
- Người dùng cuối: Hành vi, nhu cầu của khách hàng khi sử dụng website du lịch.
- Các website du lịch hiện có: Học hỏi từ các website du lịch thành công để áp dụng best practices.

1.5. Phạm vi nghiên cứu

Phạm vi nghiên cứu của đề tài được giới hạn:

- Phạm vi chức năng: Tập trung vào các chức năng cốt lõi: giới thiệu điểm đến, tìm kiếm và đặt tour, đặt phòng khách sạn, đặt vé máy bay, thuê xe, hỗ trợ visa và tư vấn du lịch.

- Phạm vi công nghệ: Sử dụng HTML5, CSS3, JavaScript thuần và thư viện Font Awesome. Không sử dụng framework phức tạp hoặc backend server.
- Phạm vi dữ liệu: Sử dụng dữ liệu tĩnh được định nghĩa sẵn trong code, không kết nối với cơ sở dữ liệu thực tế.
- Phạm vi đối tượng người dùng: Khách hàng có nhu cầu du lịch trong nước và quốc tế, có khả năng sử dụng internet và thiết bị di động.

1.6. Phương pháp nghiên cứu

Đề tài sử dụng các phương pháp nghiên cứu:

- Nghiên cứu tài liệu: Thu thập, phân tích tài liệu về công nghệ web, thiết kế giao diện và các website du lịch hiện có.
- Khảo sát và phân tích: Khảo sát các website du lịch phổ biến để phân tích ưu nhược điểm, học hỏi tính năng tốt.
- Thiết kế và phát triển: Áp dụng quy trình phát triển phần mềm từ phân tích yêu cầu, thiết kế, lập trình, kiểm thử đến triển khai.
- Thử nghiệm: Kiểm thử website trên nhiều trình duyệt và thiết bị để đảm bảo tính tương thích.
- Đánh giá và cải tiến: Thu thập phản hồi, đánh giá kết quả và cải tiến website.

1.7. Ý nghĩa thực tiễn của đề tài

Đề tài có các ý nghĩa thực tiễn:

- Đối với ngành du lịch: Cung cấp giải pháp công nghệ hỗ trợ các công ty du lịch quảng bá dịch vụ, tiếp cận khách hàng hiệu quả, tăng doanh thu.
- Đối với người dùng: Truy cập thông tin du lịch mọi lúc mọi nơi, tiết kiệm thời gian và chi phí, hỗ trợ đưa ra quyết định đúng đắn.

- Đối với giáo dục: Rèn luyện kỹ năng phân tích, thiết kế và phát triển phần mềm web, tích lũy kinh nghiệm thực tế.
- Đối với công nghệ: Ứng dụng thành công các công nghệ web hiện đại, tạo ra sản phẩm có tính ứng dụng cao.
- Đối với xã hội: Góp phần thúc đẩy phát triển du lịch, quảng bá hình ảnh các điểm đến.

1.8. Cấu trúc báo cáo

Báo cáo đồ án được cấu trúc thành các chương:

Chương 1: GIỚI THIỆU ĐỀ TÀI VÀ PHÂN TÍCH YÊU CẦU

Chương 2: CƠ SỞ LÝ THUYẾT VÀ THỰC NGHIỆM

Chương 3: THIẾT KẾ VÀ TRIỂN KHAI WEBSITE

Chương 4: KIỂM THỬ, TỐI ƯU HÓA VÀ TRIỂN KHAI

Chương 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ THỰC NGHIỆM

2.1. Cơ sở lý thuyết

2.1.1. HTML

2.1.1.1. Khái niệm

HTML (HyperText Markup Language) là ngôn ngữ đánh dấu chuẩn được sử dụng để xây dựng và mô tả cấu trúc của các trang web. HTML tổ chức nội dung thông qua các phần tử (elements), trong đó mỗi phần tử bao gồm thẻ mở, nội dung và thẻ đóng. Các phần tử này giúp trình duyệt hiểu và hiển thị nội dung theo đúng định dạng, đồng thời gắn nhãn cho từng phần văn bản như tiêu đề, đoạn văn, hình ảnh hoặc liên kết.

Cấu trúc cơ bản của một trang HTML có thể được mô tả như sau:



Hình 2.1. Cấu trúc trang html

2.1.1.2. Các thành phần ngữ nghĩa (Semantic Elements)

Semantic elements (các phần tử ngữ nghĩa) là những phần tử HTML có khả năng mô tả ý nghĩa và vai trò của nội dung bên trong một cách rõ ràng đối với cả trình duyệt và lập trình viên. Khác với các phần tử không mang tính ngữ nghĩa như `<div>` hay `<span>`, các phần tử ngữ nghĩa cho biết chính xác nội dung mà chúng biểu diễn, giúp cấu trúc trang web rõ ràng, dễ đọc và hỗ trợ tốt hơn cho SEO cũng như các công cụ hỗ trợ truy cập.

Các thẻ semantic trong HTML đóng vai trò quan trọng trong việc xác định rõ cấu trúc và ý nghĩa của từng khu vực nội dung trên trang web. Cụ thể:

- Thẻ <header>: Được sử dụng để mô tả phần mở đầu của trang web hoặc của một mục nội dung. Thành phần này thường bao gồm tiêu đề, khẩu hiệu, logo hoặc thông tin giới thiệu, góp phần định hướng nội dung ngay từ đầu.
- Thẻ <footer>: Đại diện cho phần kết thúc của trang hoặc một khu vực nội dung. Tại đây thường chứa thông tin bản quyền, tác giả, liên hệ hoặc các liên kết phụ trợ, giúp hoàn thiện bố cục tổng thể của tài liệu.
- Thẻ <nav>: Xác định khu vực điều hướng của website, bao gồm các liên kết quan trọng giúp người dùng di chuyển giữa các phần hoặc các trang. Thẻ này hỗ trợ cải thiện khả năng truy cập và trải nghiệm người dùng.
- Thẻ <aside>: Dùng để biểu diễn nội dung bổ trợ, không thuộc phần nội dung chính nhưng có liên quan gián tiếp, thường xuất hiện dưới dạng thanh bên (sidebar) hoặc khối thông tin chú thích.
- Thẻ <main>: Xác định khu vực nội dung chính của trang web, nơi chứa thông tin quan trọng và có giá trị nhất của tài liệu. Mỗi trang chỉ nên có một thẻ <main> để đảm bảo tính rõ ràng và hỗ trợ các công cụ hỗ trợ truy cập.
- Thẻ <article>: Được dùng cho các nội dung độc lập, có thể tồn tại riêng biệt như bài báo, bài blog, bài đăng diễn đàn hoặc thẻ sản phẩm. Nội dung trong <article> có tính tự hoàn chỉnh và có thể được phân phối độc lập.

- Thẻ `<section>`: Biểu diễn từng phần nội dung có chủ đề rõ ràng trong tài liệu. Mỗi section thường đi kèm tiêu đề, dùng để chia nhỏ tài liệu thành các nhóm nội dung logic, giúp tổ chức cấu trúc trang một cách khoa học.



Hình 2.2. Layout cơ bản web

2.1.1.3. Khối HTML và các phần tử nội tuyến(Inline và Block-Level Elements)

Trong HTML, các phần tử được phân chia thành hai nhóm chính là phần tử dạng khối (block-level) và phần tử dạng dòng (inline). Sự phân loại này dựa trên cách thức mỗi phần tử chiếm không gian và tương tác với các phần tử khác trong tài liệu.

Các phần tử block-level luôn bắt đầu trên một dòng mới và chiếm toàn bộ chiều ngang của phần tử chứa nó. Nhờ đặc điểm này, các phần tử block thường xếp chồng theo trục dọc, tạo thành cấu trúc từng khối nội dung rõ ràng. Đồng thời, phần tử block cho phép thiết lập chiều rộng, chiều cao và khoảng cách (margin) trên/dưới. Một số phần tử block thông dụng gồm: `div`, `p`, `h1–h6`, `ul`, `ol`, `li`, `table`.

Ngược lại, phần tử inline không tạo dòng mới mà nằm trong dòng văn bản hiện tại. Chúng chỉ chiếm diện tích tối thiểu cần thiết để hiển thị nội dung và có thể xuất hiện cạnh các phần tử inline khác. Các phần tử inline không hỗ trợ thiết lập chiều rộng, chiều cao hoặc margin theo trục dọc. Những phần tử inline phổ biến gồm: `span`, `a`, `img`, `strong`, `small`.

Sự khác biệt cốt lõi giữa hai loại phần tử này nằm ở cách chúng chiếm không gian và phân bố trong tài liệu. Nắm vững sự khác biệt này giúp người phát triển web xây dựng bố cục chính xác và kiểm soát hành vi hiển thị của từng thành phần trong trang web.

2.1.1.4. HTML Table

Thẻ `<table>` được sử dụng để biểu diễn dữ liệu dạng bảng trong HTML, bao gồm các hàng và cột. Cấu trúc cơ bản của bảng thường bao gồm các thẻ con như `<tr>` (table row) để tạo hàng, `<th>` (table header) để tạo ô tiêu đề, và `<td>` (table data) để tạo ô dữ liệu. Việc sử dụng bảng giúp trình bày thông tin có tính hệ thống, dễ quan sát và phù hợp cho các nội dung dạng thống kê hoặc liệt kê. Trong thiết kế hiện đại, bảng chủ yếu được dùng cho dữ liệu, không còn được dùng cho mục đích bố cục giao diện như trước.

2.1.1.5. HTML Form

Thẻ `<form>` là thành phần quan trọng dùng để thu thập dữ liệu từ người dùng. Bên trong form có thể chứa nhiều loại trường nhập liệu như `<input>`, `<textarea>`, `<select>`, và các nút gửi (`<button>`). Dữ liệu nhập vào sẽ được gửi đến máy chủ thông qua các phương thức như GET hoặc POST. Thẻ form đóng vai trò then chốt trong các ứng dụng web tương tác, đặc biệt trong các chức năng đăng ký, đăng nhập, phản hồi người dùng, hoặc tìm kiếm thông tin. Việc thiết kế form cần đảm bảo tính trực quan, dễ sử dụng và tuân thủ các tiêu chuẩn về khả năng truy cập.

2.1.1.6. Các thẻ hay sử dụng.

Một số thẻ HTML được sử dụng thường xuyên trong quá trình xây dựng giao diện web bao gồm:

- `<div>`: Thẻ phân chia khối dùng để nhóm các phần tử lại với nhau và tạo bố cục. Đây là thẻ block-level phổ biến nhất, được sử dụng linh hoạt kết hợp với CSS.
- `<span>`: Thẻ inline dùng để nhóm các nội dung nhỏ bên trong dòng, thường áp dụng khi cần thêm kiểu dáng cho một đoạn văn bản cụ thể.
- `<img>`: Dùng để chèn hình ảnh vào trang web, hỗ trợ nhiều định dạng như JPG, PNG, SVG.

- <a>: Thẻ liên kết, cho phép điều hướng giữa các trang hoặc đến các tài nguyên khác.
- <button>: Thẻ tạo nút tương tác, thường xuất hiện trong form hoặc các thao tác người dùng.
- <input>: Thẻ cung cấp các trường nhập liệu đa dạng như text, email, password, checkbox, radio,...

2.1.2. CSS

2.1.2.1. Bộ chọn (CSS Selectors)

Là thành phần dùng để xác định các phần tử HTML mà quy tắc CSS sẽ tác động. Nhờ bộ chọn, người lập trình có thể áp dụng kiểu dáng một cách chính xác và có kiểm soát. Các bộ chọn CSS thường được phân thành năm nhóm chính: bộ chọn đơn giản, bộ chọn kết hợp, bộ chọn lớp giả, bộ chọn phần tử giả và bộ chọn dựa trên thuộc tính.

Bộ chọn phần tử cho phép chọn các thẻ HTML dựa trên tên thẻ. Ví dụ, bộ chọn `p` sẽ áp dụng quy tắc CSS cho tất cả các phần tử `<p>` trên trang. Điều này thường được sử dụng để định dạng mặc định cho một loại thẻ nhất định, chẳng hạn như căn giữa hoặc đổi màu văn bản.

```
p {
    text-align: center;
    color: red;
}
```

Bộ chọn ID sử dụng thuộc tính id của phần tử để chọn duy nhất một phần tử trên trang. ID có tính duy nhất, do đó bộ chọn này phù hợp cho các đối tượng đặc biệt cần định dạng riêng. Cú pháp sử dụng ký hiệu `#` trước tên ID.

```
#para1 {
    text-align: center;
    color: red;
}
```

} Lưu ý: Tên ID không được bắt đầu bằng số.

Bộ chọn lớp sử dụng thuộc tính class để chọn một hoặc nhiều phần tử có cùng lớp định dạng. Ký hiệu được sử dụng là dấu chấm trước tên lớp.

```
.center {  
    text-align: center;  
    color: red;  
}
```

Người lập trình cũng có thể kết hợp tên thẻ và tên lớp để chỉ định rằng chỉ các phần tử thuộc loại nhất định mới được áp dụng kiểu:

```
p.center {  
    text-align: center;  
    color: red;  
}
```

Một phần tử có thể mang nhiều lớp, cho phép tái sử dụng và kết hợp các quy tắc CSS linh hoạt.

Bộ chọn toàn cục (*) áp dụng quy tắc CSS cho tất cả các phần tử trên trang. Bộ chọn này thường được dùng để thiết lập thiết lập chung hoặc reset CSS.

```
* {  
    text-align: center;  
    color: blue;  
}
```

Bộ chọn nhóm giúp gom nhiều bộ chọn có cùng định dạng vào một quy tắc duy nhất nhằm giảm độ dài mã và tăng tính dễ đọc. Các bộ chọn được phân tách bằng dấu phẩy:

```
h1, h2, p {  
    text-align: center;  
    color: red;  
}
```

Việc nhóm các bộ chọn làm mã CSS gọn gàng, tránh trùng lặp và đảm bảo khả năng bảo trì cao hơn.

2.1.2.2. Các đơn vị đo (CSS Units)

Trong CSS, nhiều thuộc tính như *width*, *margin*, *padding*, *font-size*... sử dụng giá trị dạng độ dài (length). Mỗi giá trị độ dài được biểu diễn bằng một số và đơn vị theo sau (ví dụ: 16px, 1.5rem). Các đơn vị trong CSS được chia thành hai nhóm chính: đơn vị tuyệt đối (absolute units) và đơn vị tương đối (relative units).

Đơn vị tuyệt đối có kích thước cố định và không thay đổi theo kích thước màn hình hay các yếu tố bên ngoài. Vì không linh hoạt, nhóm đơn vị này thường không được khuyến nghị cho thiết kế web, nhưng phù hợp cho các tài liệu in ấn hoặc khi đã biết rõ môi trường hiển thị.

Sử dụng pixel giúp kiểm soát chính xác kích thước chữ, nhưng hạn chế khả năng co giãn trên nhiều thiết bị khác nhau.

```
h1 { font-size: 40px; }
```

```
h2 { font-size: 30px; }
```

```
p { font-size: 16px; }
```

Lưu ý: không được đặt dấu cách giữa số và đơn vị; riêng giá trị 0 có thể bỏ đơn vị.

Một số đơn vị tuyệt đối phổ biến:

Đơn vị	Mô tả
px	Pixel (1px = 1/96 inch) – đơn vị tuyệt đối dùng nhiều nhất
cm	Centimet
mm	Milimet
in	Inch (1in = 96px = 2.54cm)
pt	Point (1pt = 1/72 inch)
pc	Pica (1pc = 12pt)

Bảng 2.1. Đơn vị tuyệt đối trong CSS

Đơn vị tương đối được tính dựa trên một giá trị tham chiếu (phần tử cha, phần tử gốc hoặc kích thước viewport). Đây là nhóm đơn vị rất phù hợp cho thiết kế web hiện đại nhờ khả năng thích ứng với nhiều kích thước màn hình.

Khi viewport có chiều rộng 500px $\rightarrow 1\text{vw} = 5\text{px}$.

Đơn vị em phụ thuộc vào kích thước chữ của phần tử cha. Ví dụ, nếu phần tử cha có `font-size = 16px` thì:

`2.5em  $\rightarrow$  40px`

`1em  $\rightarrow$  16px`

`Body { font-size: 16px; }`

`h1 { font-size: 2.5em; }`

`h2 { font-size: 1.875em; }`

`p { font-size: 1em; }`

Ưu điểm: Người dùng có thể thay đổi kích thước chữ thông qua cài đặt trình duyệt. Đơn vị rem luôn dựa trên kích thước chữ của phần tử gốc `<html>`, giúp toàn bộ giao diện dễ dàng mở rộng theo tỷ lệ đồng nhất.

Trong đa số trình duyệt, `<html>` mặc định có `font-size = 16px`, vì vậy:

`1rem = 16px`

`html { font-size: 16px; }`

`2.5rem = 40px`

`h1 { font-size: 2.5rem; }`

`1.875rem = 30px`

`h2 { font-size: 1.875rem; }`

`p { font-size: 1rem; }`

Đơn vị rem đặc biệt hữu ích để xây dựng giao diện *scalable* và *responsive* vì chỉ cần thay đổi `font-size` của `html`, toàn bộ bố cục sẽ tự động điều chỉnh theo.

Một số đơn vị tương đối phổ biến:

Đơn vị	Mô tả
em	Tương đối với font-size của phần tử cha
ex	Tương đối với chiều cao x-height của font hiện tại (ít dùng)
ch	Tương đối với chiều rộng ký tự “0”
fr	Đơn vị phân chia trong CSS Grid
rem	Tương đối với font-size của phần tử <html>
vw	1% chiều rộng của viewport
vh	1% chiều cao của viewport
vmin	1% của cạnh nhỏ hơn viewport
vmax	1% của cạnh lớn hơn viewport
%	Tương đối theo kích thước phần tử cha

Bảng 2.2. Đơn vị tương đối trong CSS

2.1.2.3. Kích thước (Dimensions)

Trong CSS, các thuộc tính height và width được sử dụng để xác định chiều cao và chiều rộng của một phần tử. Bên cạnh đó, thuộc tính max-width cho phép thiết lập giới hạn tối đa cho chiều rộng, giúp giao diện hiển thị linh hoạt hơn trên nhiều kích thước màn hình.

Thuộc tính height và width quy định kích thước của *vùng nội dung* (content box) của phần tử. Các giá trị này không bao gồm padding, border và margin. Do đó, kích thước thực tế của phần tử có thể lớn hơn giá trị được khai báo nếu có thêm khoảng đệm hoặc viền.

Các giá trị thường sử dụng

auto: giá trị mặc định, trình duyệt tự tính toán kích thước.

length: giá trị tuyệt đối hoặc tương đối như px, cm, em,...

?: tính theo tỷ lệ phần trăm của phần tử chứa (containing block).

initial: trả về giá trị mặc định.

inherit: kế thừa giá trị từ phần tử cha.

Ví dụ minh họa

Thiết lập chiều cao 200px và chiều rộng 50% cho phần tử <div>:

```
div {  
    height: 200px;  
    width: 50%;  
    background-color: powderblue;  
}
```

Ví dụ khác với kích thước cố định:

```
div {  
    height: 100px;  
    width: 500px;  
    background-color: powderblue;  
}
```

Lưu ý: height và width chỉ áp dụng cho vùng nội dung, không bao gồm padding, border và margin.

Thuộc tính max-width được dùng để thiết lập chiều rộng tối đa mà phần tử được phép đạt tới. Khi kích thước trình duyệt nhỏ hơn giá trị width cố định, phần tử có thể vượt khỏi màn hình khiến trình duyệt phải hiển thị thanh cuộn ngang. Việc dùng max-width giúp phần tử thu nhỏ lại một cách tự nhiên, cải thiện khả năng hiển thị trên các kích thước màn hình nhỏ.

Các giá trị phổ biến

length: đặt giới hạn tối đa theo đơn vị px, cm, v.v.

?: giới hạn tối đa theo tỷ lệ phần trăm kích thước phần tử chứa.

none: giá trị mặc định, không giới hạn chiều rộng tối đa.

Ví dụ minh họa

So sánh hai cách thiết lập chiều rộng:

```
.div1 {  
    max-width: 500px;  
    background-color: powderblue;  
}  
  
.div2 {  
    width: 500px;  
    background-color: powderblue;  
}
```

Trong khi .div2 luôn có width = 500px, thì .div1 sẽ co lại nếu màn hình nhỏ hơn 500px, giúp giao diện linh hoạt hơn.

Trường hợp sử dụng đồng thời width và max-width

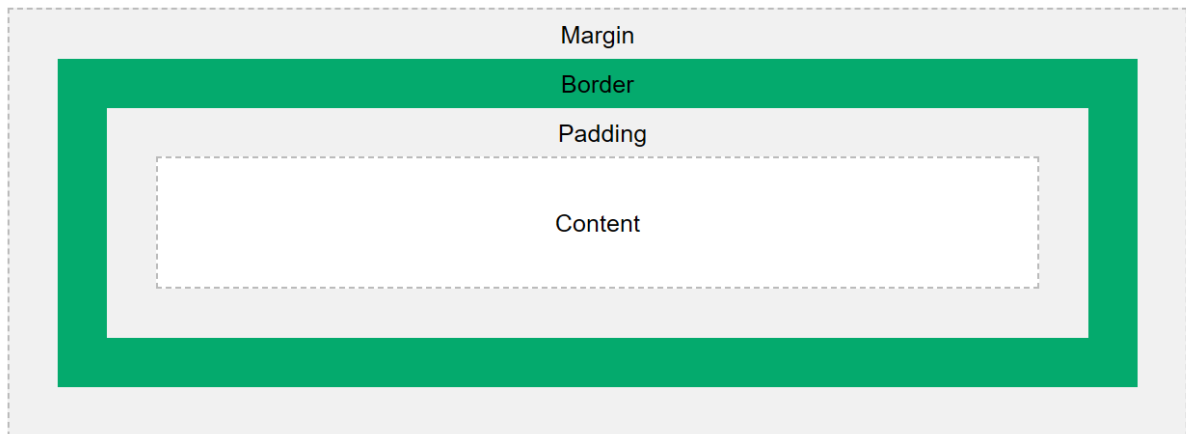
Khi một phần tử có cả width và max-width, nếu width lớn hơn max-width, trình duyệt sẽ ưu tiên giá trị của max-width.

```
.div1  
  
width: 100%;  
  
max-width: 900px;  
  
background-color: powderblue;
```

Cách thiết lập này thường được sử dụng để tạo giao diện vừa toàn màn hình vừa giới hạn chiều rộng tối đa, phù hợp với thiết kế hiện đại.

2.1.2.4. Mô hình hộp (CSS Box Model)

Trong thiết kế và bố cục trang web, khái niệm CSS Box Model được sử dụng để mô tả cấu trúc hiển thị của mọi phần tử HTML. Theo mô hình này, mỗi phần tử trên trang được biểu diễn dưới dạng một hộp gồm bốn lớp xếp chồng từ trong ra ngoài: content, padding, border và margin.



Hình 2.3. Mô hình hộp CSS

Cấu trúc của Box Model

- Content: Là vùng chứa nội dung chính của phần tử, bao gồm văn bản, hình ảnh hoặc các nội dung hiển thị khác.
- Padding: Tạo khoảng trống bao quanh nội dung. Padding luôn trong suốt và làm tăng tổng diện tích chiếm chỗ của phần tử.
- Border: Là đường viền bao quanh vùng padding và nội dung. Border có thể điều chỉnh độ dày, màu sắc và kiểu viền.
- Margin: Tạo khoảng cách giữa phần tử và các phần tử xung quanh. Margin trong suốt và không ảnh hưởng đến kích thước hộp nội bộ, nhưng ảnh hưởng đến khoảng không gian tổng thể mà phần tử chiếm trên trang.

Box Model cho phép người thiết kế điều chỉnh không gian và viền xung quanh phần tử, qua đó kiểm soát bố cục giao diện một cách linh hoạt.

Một ví dụ thể hiện rõ cấu trúc của Box Model:

```
div {  
  
    width: 300px;  
  
    border: 15px solid green;
```

```
padding: 50px;
margin: 20px;
}
```

Trong ví dụ này, phần tử <div> có:

- vùng nội dung rộng 300px,
- padding 50px,
- border dày 15px,
- margin 20px.

Để thiết lập chính xác chiều rộng và chiều cao của phần tử, cần hiểu rằng các thuộc tính width và height chỉ áp dụng cho *vùng nội dung (content box)*. Khi tính toán kích thước tổng thể, cần cộng thêm padding và border.

Ví dụ:

```
div {
    width: 320px;
    height: 50px;
    padding: 10px;
    border: 5px solid gray;
    margin: 0;
}
```

Phần tử trên có kích thước thực tế:

➤ Tổng chiều rộng:

$320\text{px (content)} + 20\text{px (padding trái + phải)} + 10\text{px (border trái + phải)} = 350\text{px}$

➤ Tổng chiều cao:

$50\text{px (content)} + 20\text{px (padding trên + dưới)} + 10\text{px (border trên + dưới)} = 80\text{px}$

- Công thức tính kích thước thực tế

Tổng chiều rộng của phần tử:

$$\text{Total width} = \text{width} + \text{padding-left} + \text{padding-right} + \text{border-left} + \text{border-right}$$

Tổng chiều cao của phần tử:

$$\text{Total height} = \text{height} + \text{padding-top} + \text{padding-bottom} + \text{border-top} + \text{border-bottom}$$

Lưu ý: Thuộc tính margin ảnh hưởng đến tổng không gian phần tử chiếm trên trang, nhưng không được tính vào kích thước thực tế của hộp. Kích thước thực tế dừng lại ở viền (border).

2.1.2.5. Đường viền (Borders)

Trong CSS, đường viền (border) là thành phần quan trọng của mô hình hộp (box model), được sử dụng để tạo ranh giới trực quan xung quanh phần tử HTML. Việc xác định đường viền giúp tăng khả năng nhận diện khu vực nội dung, phân tách bố cục và hỗ trợ tạo điểm nhấn trong thiết kế giao diện người dùng.

Một đường viền trong CSS được hình thành từ ba thuộc tính cơ bản:

- Độ dày (border-width): Xác định độ rộng của đường viền.
- Kiểu đường viền (border-style): Chỉ định dạng thể hiện của đường viền.
- Màu sắc (border-color): Xác định màu hiển thị.

Ngoài ra, CSS cho phép thiết lập đường viền cho cả bốn phía hoặc từng phía riêng biệt, bao gồm: border-top, border-right, border-bottom, border-left. Trong trường hợp cần khai báo đồng thời cả ba thành phần, có thể sử dụng cú pháp rút gọn: border: width style color;

Thuộc tính border-style giữ vai trò quyết định việc một đường viền có được hiển thị hay không, đồng thời xác định hình dạng của đường viền. Đây là thuộc tính bắt buộc phải có để các thuộc tính đường viền khác phát huy tác dụng.

Các giá trị được hỗ trợ bao gồm:

- *dotted*: Hiển thị đường viền dạng chấm.
- *dashed*: Hiển thị đường viền dạng nét đứt.
- *solid*: Hiển thị đường viền liền mạch.
- *double*: Hiển thị đường viền kép.
- *groove*: Hiển thị đường viền có rãnh dạng 3D
- *ridge*: Hiển thị đường viền dạng gờ 3D; hiệu ứng phụ thuộc vào màu đường viền.
- *inset*: Hiển thị đường viền dạng chìm xuống
- *outset*: Hiển thị đường viền dạng nổi lên
- *none*: Không hiển thị đường viền.
- *hidden*: Ẩn đường viền, thường được sử dụng trong bố cục bảng.

2.1.2.6. Thuộc tính hiển thị (Display Property)

Thuộc tính display là một trong những thuộc tính quan trọng nhất trong CSS vì nó quyết định cách một phần tử HTML tham gia vào bố cục trang web. Cụ thể, thuộc tính này xác định liệu phần tử được hiển thị dưới dạng phần tử khối (block-level element) hay phần tử nội tuyến (inline element).

Mỗi thẻ HTML đều có một giá trị hiển thị mặc định, chủ yếu thuộc một trong hai loại: block hoặc inline. Tuy nhiên, thuộc tính *display* cho phép thay đổi hành vi này nhằm đáp ứng các yêu cầu bố cục khác nhau trong thiết kế giao diện.

Phần tử cấp khối luôn bắt đầu trên một dòng mới và mặc định chiếm toàn bộ chiều rộng khả dụng theo phương ngang. Điều này khiến chúng trở thành các khối độc lập trong bố cục.

Các phần tử HTML phổ biến có kiểu hiển thị mặc định dạng block gồm:

- | | |
|---------------|------------|
| • <div> | • <form> |
| • <h1> – <h6> | • <header> |
| • <p> | • <footer> |

Phần tử nội tuyến không bắt đầu trên dòng mới và chỉ chiếm không gian đúng bằng nội dung của nó. Do đó, chúng thường được sử dụng để bao quanh văn bản hoặc các thành phần nhỏ.

Một số phần tử inline tiêu biểu:

- `<span>`
- `<a>`
- `<img>`

Thuộc tính `display` hỗ trợ nhiều giá trị để phục vụ các mục tiêu bố cục khác nhau. Bảng dưới đây trình bày các giá trị thông dụng:

Giá trị	Mô tả
<code>inline</code>	Hiển thị phần tử dưới dạng nội tuyến.
<code>block</code>	Hiển thị phần tử như một khối độc lập.
<code>contents</code>	Loại bỏ phần tử bao ngoài, chỉ giữ lại các phần tử con trong DOM.
<code>flex</code>	Biến phần tử thành khung chứa flex ở cấp độ block.
<code>grid</code>	Biến phần tử thành khung chứa grid ở cấp độ block.
<code>inline-block</code>	Kết hợp đặc điểm inline và block: hiển thị như inline nhưng có thể đặt <code>width</code> , <code>height</code> , <code>margin</code> , <code>padding</code> .
<code>none</code>	Ẩn hoàn toàn phần tử, không chiếm diện tích trong bố cục.

Bảng 2.3. Giá trị thông dụng của thuộc tính `Display`

Khi áp dụng `display: none`, phần tử sẽ bị loại bỏ khỏi luồng tài liệu và không chiếm bất kỳ diện tích nào. Giá trị này thường được sử dụng trong tương tác bằng JavaScript để ẩn/hiện nội dung.

Ví dụ minh họa:

```
#panel {  
    display: none;  
}
```

Ẩn – hiện bằng JavaScript:

```
function myFunction() {
```

```
document.getElementById("panel").style.display = "block";  
  
}
```

Hoặc chuyển đổi (toggle):

```
function myFunction() {  
  
    var x = document.getElementById("panel");  
  
    if (x.style.display === "none") {  
        x.style.display = "block";  
    } else {  
        x.style.display = "none";  
    }  
  
}
```

2.1.2.7. Định vị phần tử (Positioning)

CSS Positioning là kỹ thuật dùng để kiểm soát vị trí của các phần tử trong trang web. Nhờ thuộc tính này, lập trình viên có thể thay đổi hoặc ghi đè luồng bố cục thông thường của tài liệu HTML, từ đó tùy chỉnh cách phần tử xuất hiện trên giao diện người dùng.

Thuộc tính cốt lõi trong cơ chế định vị là `position`, kết hợp cùng các thuộc tính `top`, `bottom`, `left`, `right` để xác định vị trí cuối cùng của phần tử.

Thuộc tính `position` xác định loại định vị mà phần tử sẽ sử dụng. Giá trị của `position` bao gồm: *static (mặc định)*, *relative*, *fixed*, *absolute*, *sticky*. Các giá trị này quyết định cách phần tử tương tác với luồng tài liệu cũng như các phần tử xung quanh.

- *Static*: Đây là kiểu định vị mặc định cho tất cả phần tử HTML. Phần tử có `position: static` luôn tuân theo luồng bố cục bình thường của trang và không bị ảnh hưởng bởi các thuộc tính `top`, `bottom`, `left` và `right`.

Ví dụ CSS:

```
div.static {  
  
    position: static;
```

```
border: 3px solid #73AD21;  
}
```

- Relative: được định vị tương đối so với vị trí ban đầu trong luồng tài liệu. Khi áp dụng top, left, bottom hoặc right, phần tử sẽ dịch chuyển khỏi vị trí gốc nhưng không làm thay đổi bố cục của các phần tử khác; khoảng trống ban đầu của nó vẫn được giữ nguyên.

Ví dụ:

```
div.relative {  
    position: relative;  
    left: 30px;  
    border: 3px solid #73AD21;
```

- Fixed: được định vị tương đối so với khung nhìn trình duyệt (viewport). Do đó, nó luôn giữ nguyên vị trí kể cả khi người dùng cuộn trang. Phần tử fixed không chiếm không gian trong luồng tài liệu và không để lại khoảng trống tại vị trí ban đầu.

Ví dụ:

```
div.fixed {  
    position: fixed;  
    bottom: 0;  
    right: 0;  
    width: 300px;  
    border: 3px solid #73AD21;  
}
```

- Absolute: được định vị tương đối so với phần tử tổ tiên gần nhất có position khác static. Nếu không tìm thấy tổ tiên phù hợp, phần tử sẽ được định vị dựa trên tài liệu gốc (document body). Các phần tử absolute bị loại bỏ khỏi luồng tài liệu, có thể chồng lên các phần tử khác.

Ví dụ:

```
div.relative {  
    position: relative;  
    width: 400px;  
    height: 200px;  
    border: 3px solid green;  
}
```

```
div.absolute {  
    position: absolute;  
    top: 80px;  
    right: 0;  
    width: 200px;  
    height: 100px;  
    border: 3px solid red;  
}
```

- Sticky: kết hợp đặc tính của relative và fixed. Ban đầu, phần tử được định vị như relative. Khi người dùng cuộn trang đến một vị trí nhất định (thỏa điều kiện của thuộc tính top, bottom, left hoặc right), phần tử sẽ chuyển sang fixed và “dính lại” tại vị trí đó. Điều kiện để sticky hoạt động là phải khai báo ít nhất một trong các thuộc tính định vị (thường dùng nhất là top).

Ví dụ:

```
div.sticky {  
    position: sticky;  
    top: 0;  
    background-color: green;  
    border: 2px solid #4CAF50; }
```

2.1.2.8. Nền (Background)

Thuộc tính nền (background) trong CSS được sử dụng để thiết lập các hiệu ứng nền cho các phần tử HTML, góp phần tạo nên tính trực quan và thẩm mỹ cho giao diện. Trong phạm vi chương này, các thuộc tính nền được đề cập bao gồm:

- background-color
- background-image
- background-repeat
- background-attachment
- background-position
- background (thuộc tính viết tắt)

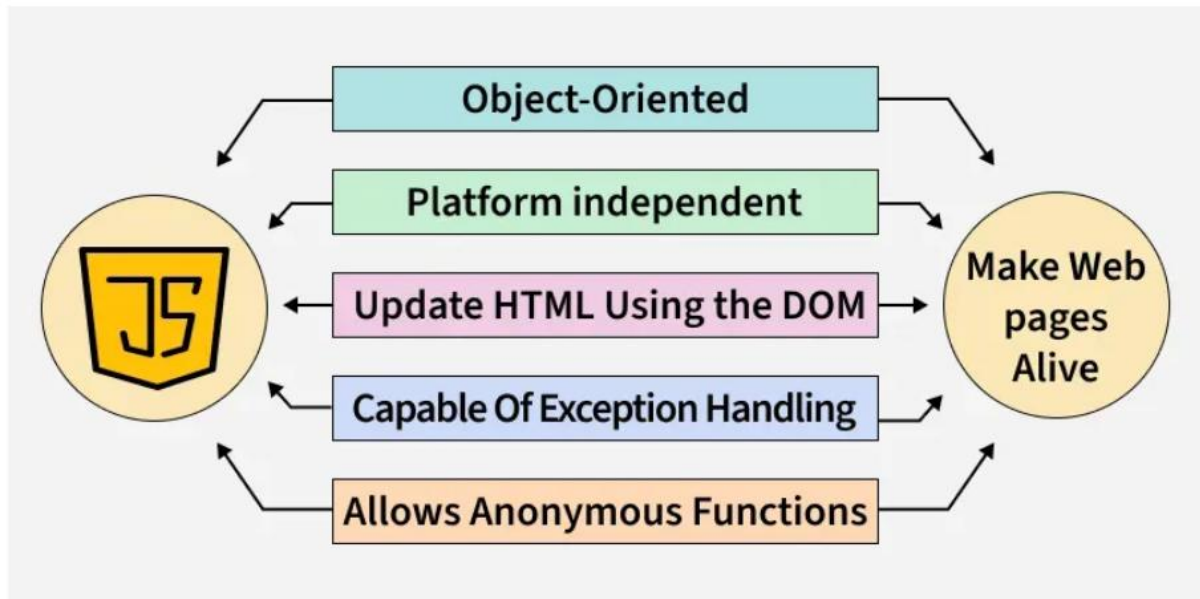
Thuộc tính background-color cho phép quy định màu nền của một phần tử. Màu sắc trong CSS có thể được xác định thông qua nhiều hình thức biểu diễn, bao gồm:

- Tên màu hợp lệ, ví dụ: "red";
- Giá trị HEX, ví dụ: #ff0000;
- Giá trị RGB, ví dụ: rgb(255, 0, 0).

2.1.3. JavaScript

2.1.3.1. Khái niệm JavaScript

JavaScript là một ngôn ngữ lập trình đa năng và kiểu động (dynamically typed), được sử dụng rộng rãi để tạo sự tương tác cho các trang web. JavaScript hỗ trợ phát triển cả phía client (trình duyệt) lẫn phía server (nhờ Node.js), đồng thời tích hợp mượt mà với HTML, CSS và thư viện chuẩn phong phú.



Hình 2.4. Các đặc điểm chính của JavaScript

JavaScript là ngôn ngữ single-threaded, thực thi từng tác vụ một, và là ngôn ngữ interpreted, nghĩa là mã lệnh được thực thi từng dòng khi chạy. Do kiểu dữ liệu được xác định trong quá trình chạy, JavaScript được xem là ngôn ngữ kiểu động.

2.1.3.2. Các tính năng của JavaScript

JavaScript sở hữu nhiều đặc điểm quan trọng khiến nó trở thành một trong những ngôn ngữ lập trình cốt lõi trong phát triển web hiện đại. Trước hết, JavaScript đóng vai trò là ngôn ngữ lập trình phía máy khách khi được thực thi trực tiếp trên trình duyệt. Điều này giúp rút ngắn thời gian phản hồi, hạn chế sự phụ thuộc vào máy chủ và mang lại trải nghiệm tương tác mượt mà cho người dùng. Bên cạnh đó, JavaScript là một ngôn ngữ đa dụng, có thể đảm nhiệm nhiều loại tác vụ khác nhau, từ xử lý các phép tính cơ bản đến xây dựng các ứng dụng quy mô lớn trên cả giao diện trình duyệt và môi trường máy chủ thông qua nền tảng Node.js.

Ngoài ra, JavaScript hoạt động theo cơ chế dựa trên sự kiện, cho phép hệ thống phản hồi ngay lập tức trước những tác động từ phía người dùng như thao tác nhấp chuột hoặc nhập liệu. Tính chất không đồng bộ của JavaScript cũng là một ưu điểm quan trọng, cho phép thực thi các tác vụ như truy xuất dữ liệu từ máy chủ mà không làm gián đoạn hoặc gây treo giao diện. Cuối cùng, JavaScript sở hữu một hệ sinh thái phong phú với nhiều thư viện và framework nổi bật như React, Angular và Vue.js. Nhờ đó, quá trình phát triển giao diện người dùng và ứng dụng web trở nên nhanh chóng, hiệu quả và dễ dàng mở rộng.

2.1.3.3. Các mô hình lập trình trong JavaScript

JavaScript hỗ trợ linh hoạt nhiều phong cách lập trình, trong đó nổi bật nhất là lập trình mệnh lệnh và lập trình khai báo. Lập trình mệnh lệnh tập trung vào mô tả cách thức thực hiện một tác vụ thông qua việc kiểm soát trực tiếp luồng tính toán. Cách tiếp cận này thường được triển khai dưới hình thức lập trình thủ tục hoặc lập trình hướng đối tượng, đồng thời sử dụng các cấu trúc như *async/await* để quản lý các thao tác bất đồng bộ một cách rõ ràng và có tính tuần tự hơn. Ngược lại, lập trình khai báo nhấn mạnh vào kết quả cần đạt được thay vì mô tả từng bước thực hiện. Phong cách này chú trọng vào việc mô tả logic thay vì quy trình, thường được thể hiện thông qua các biểu thức hàm hoặc hàm mũi tên, giúp mã lệnh ngắn gọn, súc tích và dễ đọc hơn. Nhờ khả năng hỗ trợ đồng thời cả hai phong cách lập trình, JavaScript trở thành một ngôn ngữ linh hoạt, đáp ứng được nhiều mô hình phát triển ứng dụng khác nhau.



Hình 2.5. Ứng dụng của JavaScript

2.1.3.4. Hạn chế của JavaScript

Mặc dù JavaScript là một ngôn ngữ lập trình có sức ảnh hưởng lớn trong lĩnh vực phát triển web, nó vẫn tồn tại một số hạn chế đáng lưu ý. Trước hết, JavaScript tiềm ẩn nhiều rủi ro về bảo mật, đặc biệt là các cuộc tấn công Cross-Site Scripting (XSS). Trong dạng tấn công này, các đoạn mã độc hại có thể được chèn vào trang web thông qua những thành phần như thẻ *img*, *object* hoặc *script*, từ đó đánh cắp dữ liệu người dùng hoặc gây ra các hành vi không mong muốn. Bên cạnh đó, JavaScript có hiệu suất thấp hơn so với nhiều ngôn ngữ truyền thống khi xử lý các tác vụ phức tạp, mặc dù đối với những thao tác đơn giản trên trình duyệt thì sự chênh lệch này thường không quá đáng kể.

Ngoài ra, việc phát triển ứng dụng nâng cao bằng JavaScript đòi hỏi lập trình viên phải nắm vững nhiều khái niệm lập trình cốt lõi, bao gồm mô hình đối tượng, tư duy bất đồng bộ và hiểu biết song song về kịch bản phía máy khách lẫn phía máy chủ. Điều này làm tăng độ phức tạp và có thể trở thành rào cản đối với người mới học. Hơn nữa, JavaScript là ngôn ngữ có hệ thống kiểu yếu, đồng nghĩa với việc các biến không yêu cầu khai báo kiểu dữ liệu rõ ràng. Việc kiểm tra kiểu không nghiêm ngặt có thể dẫn đến lỗi khó phát hiện, gây ảnh hưởng đến độ ổn định của chương trình trong những dự án lớn hoặc có nhiều thao tác dữ liệu.

2.1.4. Lý thuyết màu.

Trong lĩnh vực thiết kế web và đồ họa, các hệ màu được sử dụng phổ biến bao gồm CMYK, RGB, HSB và mã màu HEX. Mỗi hệ màu được phát triển dựa trên cơ chế pha màu khác nhau, phục vụ những mục đích thiết kế riêng biệt.

2.1.4.1. Hệ màu CMYK

Hệ màu CMYK là hệ màu sử dụng trong in ấn, được cấu thành từ bốn thành phần gồm Cyan (xanh lơ), Magenta(hồng cánh sen), Yellow(vàng) và Black(đen). Đây là hệ màu hoạt động theo cơ chế trừ màu, nghĩa là màu sắc được tạo ra dựa trên khả năng hấp thụ ánh sáng của các lớp mực in. Khi các lớp màu được chồng lên nhau, lượng ánh sáng bị triệt tiêu thay đổi, từ đó hình thành các sắc độ mới. Do đặc tính phụ thuộc vào chất liệu giấy và mật độ mực, hệ màu CMYK thường cho kết quả tối hơn và ít rực rỡ hơn so với màu hiển thị trên màn hình. CMYK được sử dụng nhằm đảm bảo tính chính xác và ổn định của sản phẩm in, giúp bản thiết kế thể hiện đúng màu sắc sau khi xuất bản.

2.1.4.2. Hệ màu RGB

Hệ màu RGB được sử dụng cho các thiết bị hiển thị số như màn hình máy tính, điện thoại và TV. Hệ màu này bao gồm ba thành phần Red, Green và Blue hoạt động theo cơ chế cộng màu, nghĩa là màu sắc được tạo ra nhờ việc tăng cường cường độ ánh sáng phát ra từ ba kênh màu cơ bản. Khi ba thành phần này kết hợp ở các mức độ sáng khác nhau, chúng tạo ra toàn bộ dải màu hiển thị trên các thiết bị điện tử. Do cơ chế dựa trên ánh sáng, RGB tái tạo màu sắc rực rỡ, có độ chói cao, phù hợp cho thiết kế giao diện, thiết kế web và các sản phẩm được hiển thị trên môi trường số. Việc nắm vững hệ màu RGB giúp nhà thiết kế đảm bảo tính nhất quán giữa bản thiết kế và sản phẩm hiển thị thực tế trên màn hình.

2.1.4.3. Hệ màu HSB

Hệ màu HSB mô tả màu sắc dựa trên ba yếu tố trực quan gồm Hue(sắc độ), Saturation(độ bão hòa) và Brightness(độ sáng). Hue thể hiện sắc thái cơ bản của màu, Saturation biểu thị mức độ bão hòa hay độ tinh khiết của màu, trong khi Brightness phản ánh độ sáng tối. Nhờ cấu trúc dựa trên cách con người cảm nhận màu sắc, hệ màu HSB giúp quá trình chọn và chỉnh sửa màu trở nên trực quan và dễ kiểm soát hơn. HSB thường được tích hợp trong các phần mềm đồ họa chuyên nghiệp nhằm hỗ trợ nhà thiết

kể tạo ra hệ thống màu đồng nhất, phù hợp với mục đích thị giác và trải nghiệm người dùng.

2.1.4.4. Mã màu HEX

Mã màu HEX là dạng biểu diễn màu trong hệ thập lục phân, gồm sáu ký tự đại diện cho ba giá trị tương ứng với các kênh màu của hệ RGB. Đây là định dạng tiêu chuẩn trong thiết kế web và được sử dụng rộng rãi trong CSS nhờ sự ngắn gọn, dễ ghi nhớ và khả năng tương thích cao với các trình duyệt. Mã HEX giúp mã hóa màu sắc một cách nhất quán, đảm bảo sự đồng bộ giữa các môi trường thiết kế và lập trình. Với tính chính xác và sự thuận tiện khi áp dụng, mã HEX trở thành định dạng phổ biến trong xây dựng giao diện người dùng và hệ thống nhận diện thương hiệu trên nền tảng số.

2.1.5. Bố cục và kiểu chữ.

2.1.5.1. Tạo kiểu văn bản (Styling text)

Trong CSS, tạo kiểu văn bản là một thành phần quan trọng nhằm đảm bảo khả năng đọc, truyền đạt thông tin và tính thẩm mỹ của giao diện. Các thuộc tính tạo kiểu văn bản bao gồm lựa chọn phông chữ, kích thước chữ, độ đậm, màu sắc, chiều cao dòng và cách căn chỉnh. Việc sử dụng hợp lý *font-family* giúp đảm bảo tính thống nhất và phù hợp với chủ đề thiết kế, trong khi *font-size* và *line-height* quyết định mức độ thoải mái khi đọc. Bên cạnh đó, các thuộc tính như *text-align*, *text-transform* và *letter-spacing* cho phép điều chỉnh bố cục và nhịp điệu của văn bản. Tạo kiểu văn bản đúng cách không chỉ nâng cao tính dễ đọc mà còn góp phần xây dựng nhận diện thương hiệu và trải nghiệm người dùng nhất quán trong toàn bộ giao diện.

2.1.5.2. Bố cục (Flexbox)

Flexbox (Flexible Box Module) là một mô hình bố cục hiện đại trong CSS được thiết kế nhằm cải thiện khả năng sắp xếp các phần tử theo trục ngang hoặc dọc một cách linh hoạt. Flexbox đặc biệt phù hợp cho những bố cục đơn chiều, nơi các phần tử cần tự động giãn, co hoặc phân chia không gian một cách hợp lý dựa trên kích thước màn hình. Bằng việc sử dụng các thuộc tính như *display: flex*, *justify-content*, *align-items* và *flex-direction*, nhà thiết kế có thể dễ dàng điều chỉnh vị trí, khoảng cách và thứ tự các phần tử mà không cần tính toán thủ công bằng giá trị cố định. Tính linh hoạt cao của Flexbox

giúp tối ưu hóa giao diện trên nhiều kích thước thiết bị, đồng thời đơn giản hóa đáng kể quy trình xây dựng bố cục so với các phương pháp truyền thống.

2.1.5.3. Lưới bố cục (Grid Layout)

CSS Grid Layout là một hệ thống bố cục hai chiều mạnh mẽ, cho phép thiết kế giao diện bằng cách chia trang web thành các hàng và cột có cấu trúc rõ ràng. Khác với Flexbox vốn tập trung vào một chiều duy nhất, Grid hỗ trợ quản lý đồng thời cả chiều ngang lẫn chiều dọc, giúp tạo ra các bố cục phức tạp một cách trực quan và hợp lý. Thông qua các thuộc tính như *display: grid*, *grid-template-columns*, *grid-template-rows* và *gap*, người thiết kế có thể xác định rõ ràng kích thước, tỷ lệ và khoảng cách giữa các khu vực nội dung. Hệ thống này mang lại khả năng kiểm soát cao, tạo điều kiện để xây dựng các giao diện nhất quán, cân đối và dễ mở rộng, đặc biệt phù hợp với những trang web yêu cầu cấu trúc bố cục chặt chẽ và chuyên nghiệp.

2.1.6. Các framework và thư viện phổ biến

2.1.6.1. Khái niệm Thư viện JavaScript

Thư viện JavaScript là tập hợp các đoạn mã, hàm hoặc đối tượng được xây dựng sẵn nhằm giải quyết một số tác vụ lặp lại trong quá trình phát triển ứng dụng. Thay vì phải viết lại toàn bộ mã nguồn cho mỗi chức năng, lập trình viên có thể tái sử dụng những thành phần đã được đóng gói trong thư viện. Điều này không chỉ giúp tiết kiệm thời gian và công sức mà còn đảm bảo tính nhất quán, khả năng bảo trì và hiệu suất của ứng dụng.

Các thư viện JavaScript có phạm vi ứng dụng rộng, được thiết kế để giải quyết nhiều nhóm vấn đề khác nhau. Chúng được dùng để trực quan hóa dữ liệu thông qua biểu đồ và bản đồ, thao tác DOM một cách hiệu quả, xử lý dữ liệu có cấu trúc lớn, làm việc với cơ sở dữ liệu, tối ưu hóa biểu mẫu, tạo hiệu ứng động hoặc hiệu ứng hình ảnh, xử lý chuỗi – ngày giờ – ma trận, và xây dựng các thành phần giao diện người dùng. Ngoài những ứng dụng phổ biến này, thư viện JavaScript còn được sử dụng trong việc tạo hộp thoại tùy chỉnh, tối ưu hóa truy vấn AJAX, căn chỉnh bố cục, xây dựng điều hướng – định tuyến và hỗ trợ gỡ lỗi.

2.1.6.2. Một số Thư viện JavaScript Tiêu biểu

jQuery: là một trong những thư viện JavaScript được sử dụng rộng rãi nhất nhờ khả năng đơn giản hóa thao tác DOM, xử lý sự kiện, hoạt ảnh và các tác vụ AJAX. Được phát hành từ năm 2006, thư viện này tối ưu công việc lựa chọn phần tử bằng CSS3 selector, có dung lượng nhỏ, dễ học và tương thích với nhiều trình duyệt khác nhau. jQuery đặc biệt phù hợp trong các dự án cần thao tác DOM nhanh và tiện lợi.

React.js: là thư viện front-end mã nguồn mở nổi bật, được Facebook phát triển. Điểm mạnh lớn nhất của React nằm ở cơ chế Virtual DOM, giúp tối ưu hiệu suất và tăng tốc độ cập nhật giao diện. React tổ chức mã nguồn dựa trên các thành phần độc lập, dễ tái sử dụng và dễ kiểm thử, đồng thời hỗ trợ cú pháp JSX kết hợp JavaScript và HTML. Thư viện này hiện được sử dụng rộng rãi trong các ứng dụng web và ứng dụng di động (thông qua React Native).

D3.js: là thư viện mạnh mẽ dành cho trực quan hóa dữ liệu, hỗ trợ thao tác DOM dựa trên dữ liệu và triển khai hình ảnh tương tác với độ phức tạp cao. D3 tận dụng khả năng của SVG, HTML và CSS, cho phép tạo ra các biểu đồ, bản đồ và hoạt cảnh dữ liệu với hiệu suất tốt.

Underscore: là thư viện tiện ích cung cấp các hàm hỗ trợ lập trình chức năng. Thư viện này gồm hơn 100 hàm xử lý mảng, đối tượng và chuỗi, giúp giảm đáng kể số lượng mã cần viết trong các tác vụ lập trình thông thường.

Lodash: được xem như phiên bản nâng cấp của Underscore với hiệu năng tốt hơn và khả năng tương thích rộng. Lodash hỗ trợ mạnh mẽ các thao tác trên mảng, chuỗi, đối tượng cùng nhiều chức năng như debounce, throttle, clone, merge, phù hợp trong các hệ thống cần xử lý dữ liệu linh hoạt và ổn định.

Algolia Places: là thư viện chuyên về gợi ý địa chỉ tự động, hoạt động nhanh và chính xác nhờ dữ liệu từ OpenStreetMap. Thư viện này giúp cải thiện trải nghiệm người dùng trong biểu mẫu và các hệ thống tìm kiếm địa điểm.

Anime.js: là thư viện hỗ trợ tạo hiệu ứng động với DOM, CSS và SVG. Nhờ API đơn giản, Anime.js cho phép tạo các chuyển động phức tạp như chuỗi hoạt cảnh, trễ so le, đảo ngược hoặc đồng bộ hóa theo sự kiện.

AOS: là thư viện giúp tạo hiệu ứng xuất hiện khi cuộn trang. Đây là lựa chọn phổ biến trong các trang web một trang hoặc trang trình bày nội dung dài nhờ khả năng hoạt động mượt mà trên nhiều thiết bị.

2.1.6.3. Khái niệm framework.

Framework là một bộ khung lập trình bao gồm các đoạn code và thư viện đã được xây dựng sẵn, giúp lập trình viên phát triển ứng dụng nhanh chóng và dễ dàng hơn. Nó cung cấp các tính năng như mô hình, API và các công cụ hỗ trợ, giúp tối giản hóa quá trình tạo ra các ứng dụng web phong phú, linh hoạt. Nói cách khác, framework giống như một khung nhà đã có sẵn nền móng, bạn chỉ cần tập trung “trang trí” và hoàn thiện theo ý tưởng của mình.

Framework giúp tăng hiệu suất, mở rộng nhiều chức năng và cung cấp các thư viện sẵn có giúp các nhà phát triển không phải làm lại từ đầu. Một framework cho web bao gồm:

- **Libraries**: Thư viện là các đoạn code xây dựng sẵn cho một chức năng nào đó (Design Pattern), bạn có thể tái sử dụng chức năng đó mà không phải code lại, có nhiều tools quản lý thư viện như NPM, Composer,...
- **API**: Là phương thức trao đổi dữ liệu giữa các ứng dụng chính và ứng dụng khác.
- **Scaffolding**: Một bộ khung các quy tắc mà một framework MVC sử dụng quy hoạch cơ sở dữ liệu có thể được truy cập như thế nào.
- **AJAX**: Update thông tin lên database mà không cần load lại trang.
- **Caching**: Giúp giảm request đến máy chủ, tăng tốc độ load trang.
- **Security**: Framework xác thực và ủy quyền user.
- **Compilers** : Trình biên dịch từ code của bạn qua ngôn ngữ cho máy.

2.1.6.4. Các framework phổ biến.

Framework được chia thành nhiều loại, mỗi loại phục vụ những mục đích phát triển phần mềm khác nhau. Dưới đây là các loại framework phổ biến nhất:

- Frontend frameworks: Dùng để xây dựng giao diện người dùng cho ứng dụng web và di động. Một số framework nổi bật gồm React, Angular, Vue.js và Bootstrap.
- Backend frameworks: Hỗ trợ xây dựng logic xử lý dữ liệu và các chức năng phía máy chủ. Những ví dụ điển hình là Ruby on Rails, Django, Flask và Laravel.
- Full-stack frameworks: Cung cấp giải pháp toàn diện cho cả frontend và backend. Các framework phổ biến như MEAN Stack (MongoDB, Express, Angular, Node.js), MERN Stack (MongoDB, Express, React, Node.js) và Laravel.
- Mobile App Development frameworks: Tập trung vào việc phát triển ứng dụng di động đa nền tảng. Các framework nổi bật gồm React Native, Xamarin, Flutter và Ionic.
- Testing frameworks: Hỗ trợ kiểm thử tính năng và hiệu suất phần mềm. Một số cái tên phổ biến là Selenium, Cypress, Jest và JUnit.
- Machine Learning frameworks: Được thiết kế để phát triển và triển khai mô hình học máy. Các framework quen thuộc gồm TensorFlow, PyTorch và Scikit-learn.
- Game Development frameworks: Cung cấp công cụ phát triển trò chơi điện tử. Những framework hàng đầu trong lĩnh vực này là Unity, Unreal Engine và Godot Engine.

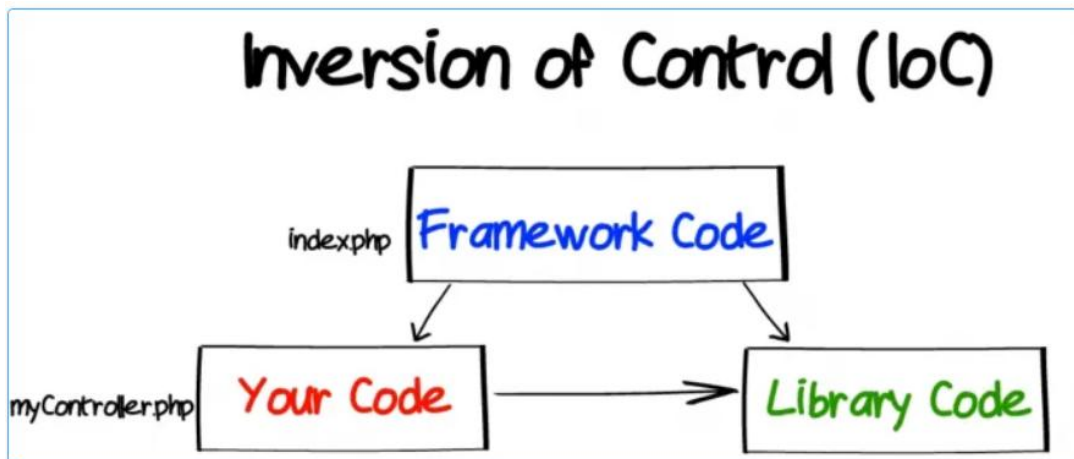
2.1.6.5. Sự khác nhau giữa framework và thư viện.

Mặc dù Library và Framework đều hỗ trợ lập trình viên trong quá trình phát triển phần mềm, nhưng chúng có sự khác biệt rõ rệt về cách tương tác và vai trò trong dự án:

Library (Thư viện): Thư viện là tập hợp các hàm hoặc công cụ được thiết kế để giải quyết những vấn đề cụ thể. Lập trình viên chủ động gọi và sử dụng các tính năng từ thư viện khi cần, đồng thời có toàn quyền kiểm soát mã nguồn. Ví dụ, jQuery giúp thao tác với các phần tử trên trang web, hoặc D3.js hỗ trợ tạo biểu đồ và đồ thị tương tác.

Framework (Khung phần mềm): Framework là một bộ khung với các quy tắc và nguyên tắc mà lập trình viên phải tuân theo khi xây dựng ứng dụng. Framework thường xác định cấu trúc dự án, cách tổ chức mã nguồn và quy định việc sử dụng các module hoặc component. Ví dụ, Angular, React, Vue.js là các framework frontend, trong khi Express.js và Django là các framework backend phổ biến.

Sự khác biệt chính nằm ở cách kiểm soát. Với Library, lập trình viên kiểm soát luồng xử lý; còn với framework, chính framework kiểm soát và gọi mã nguồn của bạn theo quy tắc đã định sẵn.



Hình 2.6. Sự khác biệt cơ bản giữa Framework và Library

2.1.7. Khái niệm về hosting, domain, sever.

2.1.7.1. Domain (Tên miền)

Tên miền (Domain) là địa chỉ định danh duy nhất của một website trên Internet, cho phép người dùng truy cập vào trang web thay vì phải ghi nhớ địa chỉ IP dạng số. Domain hoạt động như một “địa chỉ nhà” trong không gian mạng, giúp xác định vị trí của trang web một cách dễ hiểu và dễ sử dụng hơn.

Cấu trúc tên miền bao gồm nhiều cấp độ, phổ biến nhất là:

- TLD (Top-Level Domain): các phần mở rộng như .com, .net, .org, .vn,...
- SLD (Second-Level Domain): phần tên được cá nhân hoặc tổ chức đăng ký, ví dụ như trong “example.com”, phần “example” là SLD.

Tên miền giữ vai trò quan trọng trong việc xây dựng thương hiệu, tính nhận diện và khả năng truy cập của hệ thống web.

2.1.7.2. Hosting (Web Hosting)

Hosting là dịch vụ cung cấp không gian lưu trữ trên máy chủ để vận hành website. Khi một website được triển khai, toàn bộ mã nguồn, cơ sở dữ liệu, hình ảnh và tài nguyên tĩnh đều cần được lưu trữ trên một máy chủ có kết nối Internet liên tục. Dịch vụ hosting đảm nhiệm vai trò này, đảm bảo website có thể truy cập 24/7.

Một số loại hình hosting phổ biến:

- Shared Hosting: nhiều website cùng sử dụng tài nguyên chung trên một máy chủ; chi phí thấp, phù hợp dự án nhỏ.
- VPS (Virtual Private Server): máy chủ ảo hóa, cung cấp tài nguyên riêng biệt; phù hợp dự án vừa và lớn.
- Dedicated Server: máy chủ vật lý riêng biệt dành cho một khách hàng; hiệu suất cao.
- Cloud Hosting: sử dụng hạ tầng đám mây, khả năng mở rộng linh hoạt và độ ổn định cao.

Hosting là nền tảng quan trọng để website hoạt động ổn định và đảm bảo tốc độ truy cập, khả năng chịu tải cũng như tính bảo mật.

2.1.7.3. Server (Máy chủ)

Server là hệ thống máy tính có cấu hình đặc biệt, được thiết kế để cung cấp tài nguyên, dữ liệu và dịch vụ cho các máy tính khác trong mạng (gọi là client). Máy chủ có thể tồn tại dưới dạng:

- Máy chủ vật lý (Physical Server): thiết bị phần cứng chuyên dụng.
- Máy chủ ảo (Virtual Server): được tạo ra thông qua công nghệ ảo hóa.
- Máy chủ đám mây (Cloud Server): tài nguyên phân tán trên hạ tầng đám mây.

Server đảm nhiệm nhiều chức năng như:

- Lưu trữ và phân phối dữ liệu;
- Chạy các ứng dụng web;
- Quản lý cơ sở dữ liệu;

- Đảm bảo an ninh và xử lý các yêu cầu từ client.

Máy chủ chính là nền tảng vận hành của bất kỳ hệ thống web hoặc dịch vụ trực tuyến nào, giữ vai trò quyết định trong tốc độ, tính ổn định và khả năng mở rộng của hệ thống.

2.2. Khảo sát các sản phẩm tương tự

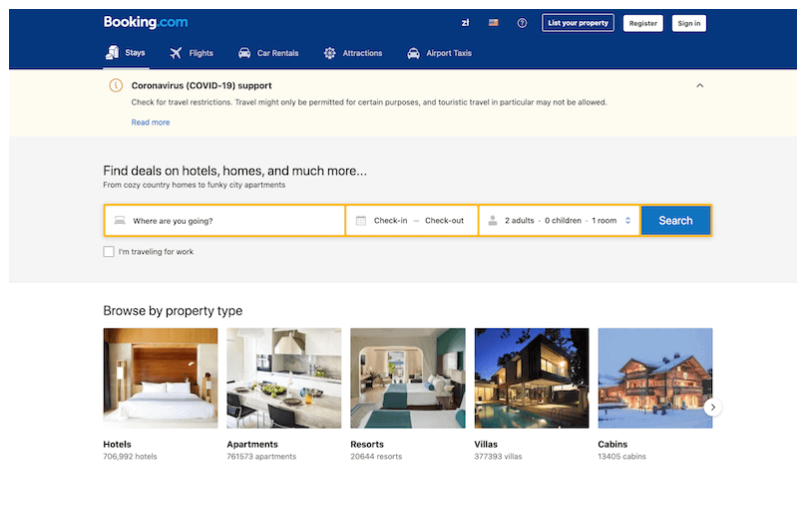
2.2.1. Phân tích các website cùng lĩnh vực

Nhằm phục vụ cho quá trình thiết kế và phát triển sản phẩm, nhóm tiến hành nghiên cứu, phân tích ba website tiêu biểu trong lĩnh vực du lịch và đặt dịch vụ trực tuyến, bao gồm: Booking.com, Traveloka.com và Vietravel.com. Đây đều là những nền tảng phổ biến, có lượng người dùng lớn và hệ thống dịch vụ đa dạng.

Website 1: Booking.com

➤ Giao diện người dùng (UI)

Booking.com sử dụng phong cách thiết kế tối giản, khoa học và tập trung vào chức năng tìm kiếm. Màu sắc chủ đạo là xanh dương đậm (003580) – màu sắc thường được sử dụng trong các hệ thống liên quan đến tài chính và dịch vụ nhằm tạo cảm giác tin cậy, an toàn cho người dùng. Bố cục trang chủ rõ ràng với phần tìm kiếm (search box) đặt ở vị trí trung tâm trong khu vực hero section. Các kết quả tìm kiếm được hiển thị theo dạng card layout, giúp người dùng dễ so sánh và lựa chọn.



Hình 2.7. Giao diện trang web Booking.com

➤ Chức năng chính

- Hệ thống tìm kiếm khách sạn, nhà nghỉ rất mạnh, hỗ trợ nhiều tham số lọc như: giá, vị trí, tiện nghi, đánh giá, khoảng cách.
- Hiện thị đánh giá, nhận xét và hình ảnh thực tế từ khách hàng trước đó, giúp tăng độ tin cậy.
- Hỗ trợ so sánh giá giữa nhiều đơn vị cung cấp dịch vụ.
- Hỗ trợ đa ngôn ngữ và đa tiền tệ, phục vụ hiệu quả cho người dùng quốc tế.

➤ Trải nghiệm người dùng (UX)

- Ưu điểm: Giao diện trực quan, thao tác tìm kiếm nhanh, thông tin đầy đủ.
- Nhược điểm: Một số người dùng mới có thể bị “ngợp” vì lượng thông tin hiển thị khá lớn; một số chức năng nâng cao chưa thực sự nổi bật.

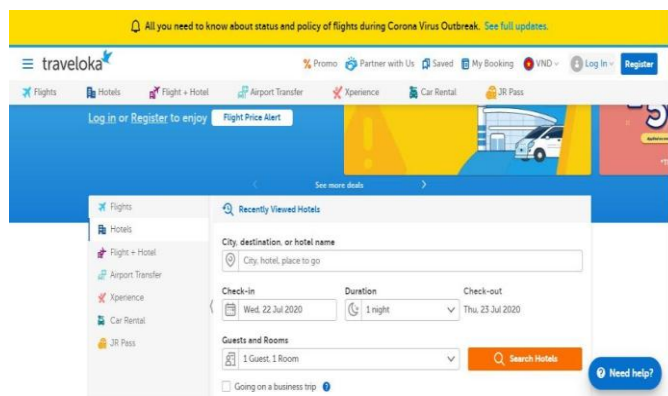
➤ Tốc độ tải trang

Booking.com có tốc độ tải nhanh nhờ kỹ thuật tối ưu hình ảnh và mã nguồn hợp lý, đồng thời sử dụng lazy loading để giảm thời gian tải ban đầu.

Website 2: Traveloka.com

➤ Giao diện người dùng (UI)

Traveloka sở hữu giao diện hiện đại với tông màu tươi sáng, nổi bật là màu cam (FF6B35) và xanh lá (00B894). Bố cục theo dạng card-based layout, giúp thông tin hiển thị gọn gàng và dễ theo dõi. Hình ảnh sử dụng có độ phân giải cao, đồng thời các icon trực quan giúp người dùng dễ nhận biết chức năng.



Hình 2.8. Giao diện trang web traveloka.com

➤ Chức năng chính

- Cung cấp nhiều dịch vụ trong cùng một nền tảng: khách sạn, vé máy bay, tour, đưa đón sân bay, hoạt động vui chơi.
- Tính năng lọc linh hoạt theo nhiều tiêu chí.
- Hỗ trợ đa dạng phương thức thanh toán: thẻ ngân hàng, ví điện tử, trả góp.
- Có ứng dụng di động được tối ưu tốt, đồng bộ với website.

➤ Trải nghiệm người dùng (UX)

- Ưu điểm: Trực quan, sinh động, điều hướng khá dễ dàng, phù hợp với người dùng trẻ.
- Nhược điểm: Một số tính năng nâng cao có thể gây khó khăn cho người mới sử dụng; thông tin ở vài mục chưa thật sự chi tiết.

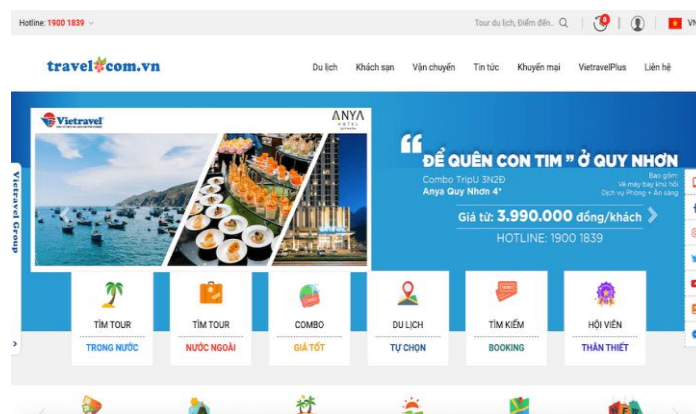
➤ Tốc độ tải trang

Tốc độ tải khá tốt, tuy nhiên trên một số trang có nhiều hình ảnh lớn, thời gian tải có thể chậm hơn so với Booking.com.

Website 3: Vietravel.com

➤ Giao diện người dùng (UI)

- Vietravel có phong cách thiết kế mang tính truyền thống, sử dụng hai màu chủ đạo là xanh dương và cam – phù hợp với bộ nhận diện thương hiệu. Tuy nhiên, layout còn hơi dày thông tin, thiếu sự tối giản và hiện đại như hai website còn lại.



Hình 2.9. Giao diện trang web vietravel.com (Nay là travel.com.vn)

➤ Chức năng chính

- Tập trung chủ yếu vào các tour du lịch trong và ngoài nước.
- Cung cấp thông tin chi tiết về lịch trình, giá cả, điều kiện tour.
- Hỗ trợ tư vấn trực tuyến và đặt tour qua nhiều kênh khác nhau.

➤ Trải nghiệm người dùng (UX)

- Ưu điểm: Thông tin đầy đủ, chi tiết, phù hợp với người cần tham khảo kỹ lưỡng.
- Nhược điểm: Điều hướng chưa tối ưu, quy trình đặt tour còn nhiều bước, chưa thực sự thuận tiện.

➤ Tốc độ tải trang

- Tốc độ tải tương đối ổn định, tuy nhiên vẫn có thể được tối ưu hơn nữa nếu cải thiện về dung lượng ảnh và mã nguồn.

2.2.2. Bảng so sánh các website

Tiêu chí đánh giá	Booking.com	Traveloka.com	Vietravel.com
Giao diện	Rất tốt – đơn giản, chuyên nghiệp, rõ ràng	Hiện đại, màu sắc tươi sáng, bắt mắt	Truyền thống, chưa thật sự hiện đại
Chức năng	Mạnh về tìm kiếm, nhiều bộ lọc	Tích hợp đa dịch vụ (vé máy bay, khách sạn)	Tập trung vào tour, thông tin chi tiết
Trải nghiệm người dùng	Dễ sử dụng nhưng hơi nhiều thông tin	Trực quan, thuận tiện, đôi lúc hơi phức tạp	Ổn định nhưng điều hướng chưa tối ưu
Tốc độ tải trang	Nhanh, được tối ưu tốt	Khá tốt, có thể chậm khi nhiều ảnh	Ổn định, có thể cải thiện thêm
Responsive	Rất tốt trên mọi thiết bị	Rất tốt, ứng dụng mobile mạnh	Tốt, nhưng chưa tối ưu hoàn toàn

Bảng 2.4. So sánh các website

2.2.3. Nhận xét và định hướng cho sản phẩm của nhóm

Qua quá trình phân tích, có thể rút ra các đặc điểm chung của các website du lịch thành công như sau:

1. Giao diện đơn giản, rõ ràng, tập trung vào chức năng chính.
2. Thanh tìm kiếm (search box) luôn được đặt ở vị trí nổi bật.
3. Card layout được sử dụng rộng rãi để hiển thị thông tin.
4. Website luôn đáp ứng tốt trên nhiều thiết bị (responsive).
5. Tốc độ tải trang được tối ưu nhằm nâng cao trải nghiệm người dùng.

Từ những kết quả phân tích trên, nhóm đề xuất định hướng thiết kế cho sản phẩm như sau:

- Xây dựng giao diện hiện đại, tinh giản, hạn chế hiển thị quá nhiều thông tin trên một màn hình.
- Sử dụng màu xanh dương làm màu chủ đạo kết hợp với màu cam tạo điểm nhấn.
- Thanh tìm kiếm được đặt ở hero section với thiết kế rõ ràng, dễ thao tác.
- Áp dụng card-based layout cho việc hiển thị điểm đến, tour và khách sạn.
- Tối ưu hiển thị trên thiết bị di động theo hướng mobile-first.
- Tối ưu dung lượng ảnh, sử dụng các định dạng mới như WebP và áp dụng lazy loading.
- Tổ chức nội dung một cách khoa học, giúp người dùng thao tác nhanh và thuận tiện.

2.3. Tổng hợp và định hướng thiết kế

2.3.1. Tóm tắt các yếu tố sẽ áp dụng cho dự án

Dựa trên kết quả khảo sát các website du lịch hiện có và nghiên cứu các nguyên lý lý thuyết về thiết kế web, đề tài đã xác định và tổng hợp các yếu tố kỹ thuật và thiết kế sẽ được áp dụng trong quá trình phát triển website. Các yếu tố này được phân loại thành bốn nhóm chính: công nghệ, thiết kế giao diện, trải nghiệm người dùng và tối ưu hiệu suất.

Đề tài sẽ sử dụng các công nghệ web chuẩn và hiện đại để xây dựng website:

- HTML5: Áp dụng các semantic elements (`<header>`, `<nav>`, `<section>`, `<article>`, `<footer>`) để cấu trúc website một cách có ý nghĩa, cải thiện khả năng truy cập (accessibility) và tối ưu hóa công cụ tìm kiếm (SEO). Sử dụng các form elements mới (`email`, `date`, `tel`) để tận dụng tính năng validation tự động của trình duyệt, nâng cao trải nghiệm người dùng.
- CSS3: Triển khai các tính năng CSS3 hiện đại bao gồm: Flexbox cho việc bố cục các thành phần giao diện (header, navigation, sections), CSS Grid cho việc sắp xếp các card điểm đến và dịch vụ theo hệ thống lưới, Media Queries để đảm bảo responsive design với các breakpoints tại 768px (tablet) và 1024px (desktop), CSS Variables (Custom Properties) để quản lý màu sắc và style một cách tập trung và dễ bảo trì, Transitions và Animations để tạo các hiệu ứng chuyển đổi mượt mà và tương tác trực quan.
- JavaScript : Sử dụng JavaScript thuần không phụ thuộc vào framework để xử lý các tương tác của người dùng, bao gồm: điều khiển menu mobile (hamburger menu), thực hiện smooth scrolling khi điều hướng, validation form phía client, và xử lý dropdown menu. Việc sử dụng Vanilla JavaScript giúp giảm thiểu dung lượng file, tăng tốc độ tải trang và phù hợp với mục tiêu học tập.
- Font Awesome: Tích hợp thư viện icon Font Awesome thông qua CDN để sử dụng các icon cho các phần tử giao diện người dùng (UI elements), menu, buttons và liên kết mạng xã hội. Font Awesome được chọn vì tính dễ sử dụng, nhẹ và có khả năng mở rộng (scalable).

Các yếu tố thiết kế được lựa chọn dựa trên nguyên tắc UI/UX và phù hợp với chủ đề du lịch:

Bảng màu sử dụng tông xanh dương làm chủ đạo với mã màu 2563eb nhằm tạo cảm giác tin cậy và chuyên nghiệp; tông xanh đậm 1e40af được dùng để tăng chiều sâu

và làm nổi bật các thành phần quan trọng; màu cam #f59e0b đóng vai trò điểm nhấn giúp tăng sự năng động và thu hút thị giác. Màu chữ được lựa chọn theo hai cấp độ gồm màu đen #030303 cho văn bản chính và xám #6b7280 cho văn bản phụ, đi kèm nền trắng #ffffff và nền xám nhạt #f9fafb nhằm giữ sự hài hòa và tạo độ tương phản phù hợp.

Về typography, dự án sử dụng nhóm font sans-serif như Segoe UI, Tahoma, Geneva và Verdana để đảm bảo khả năng đọc tốt trên màn hình kỹ thuật số. Hệ thống phân cấp chữ được thiết lập rõ ràng với kích thước H1 là 2.5rem, H2 là 2rem, H3 là 1.5rem và body text là 1rem. Line-height được điều chỉnh ở mức 1.6 cho phần nội dung nhằm tăng tính dễ đọc và 1.2 cho các tiêu đề để giữ sự gọn gàng và nhất quán thị giác.

Hệ thống bố cục được xây dựng dựa trên grid và flexbox nhằm mang lại sự linh hoạt. Grid layout được áp dụng cho danh sách điểm đến và dịch vụ, với số cột tự động thay đổi theo kích thước màn hình: ba cột trên desktop, hai cột trên tablet và một cột trên mobile. Flexbox hỗ trợ căn chỉnh header, navigation và các thành phần khác để duy trì cấu trúc gọn gàng. Bên cạnh đó, nội dung được giới hạn trong container có chiều rộng tối đa 1200px kết hợp padding 20px để tạo khoảng cách cân đối hai bên. Khoảng cách trong bố cục sử dụng đơn vị rem giúp duy trì tính nhất quán và khả năng mở rộng, đồng thời các phần tử đều được bố trí whitespace hợp lý để tránh cảm giác rối mắt.

Các yếu tố UX được thiết kế để đảm bảo website dễ sử dụng và hiệu quả:

➤ Navigation (Điều hướng):

- Menu điều hướng rõ ràng với các mục: Trang Chủ, Điểm Đến, Dịch Vụ, Liên Hệ
- Hamburger menu cho thiết bị di động với animation mượt mà
- Dropdown menu cho mục "Điểm Đến" để phân loại điểm đến trong nước và quốc tế
- Smooth scrolling khi click vào các link anchor để tạo mượt mà

➤ Search Functionality (Chức năng tìm kiếm):

- Form tìm kiếm được đặt nổi bật ở hero section để dễ nhận biết

- Input và select được thiết kế rõ ràng, dễ sử dụng
- Validation cơ bản phía client để cải thiện trải nghiệm người dùng

➤ Card-based Layout:

- Sử dụng card layout để hiển thị điểm đến và dịch vụ
- Mỗi card bao gồm: hình ảnh (tỷ lệ 16:9), tiêu đề, mô tả ngắn gọn, giá cả
- Hover effect: scale nhẹ và shadow để tạo phản hồi trực quan

➤ Responsive Design:

- Áp dụng mobile-first approach trong thiết kế
- Breakpoints tại 768px (tablet) và 1024px (desktop)
- Menu tự động chuyển sang hamburger menu trên thiết bị di động

2.3.2. Lý do chọn framework, công cụ và phong cách thiết kế cụ thể

Việc lựa chọn công nghệ, thư viện và phong cách thiết kế được thực hiện dựa trên yêu cầu của dự án, mục tiêu học tập và các chuẩn thực hành phổ biến trong thiết kế web hiện đại. Font Awesome được sử dụng vì khả năng tích hợp đơn giản thông qua CDN, dễ áp dụng vào mã nguồn và không đòi hỏi thiết kế icon thủ công. Thư viện này hoạt động hiệu quả nhờ cơ chế icon dạng font giúp giảm số lượng yêu cầu tải về, đồng thời cung cấp bộ icon đa dạng và khả năng tùy biến linh hoạt bằng CSS, phù hợp với giao diện responsive.

Phong cách thiết kế Modern – Clean được lựa chọn vì phù hợp với xu hướng hiện nay và đảm bảo tính chuyên nghiệp cho website du lịch. Giao diện tối giản giúp nội dung và hình ảnh du lịch trở nên nổi bật, trong khi bố cục dạng thẻ hỗ trợ trình bày thông tin rõ ràng, dễ so sánh và dễ mở rộng trên nhiều kích thước màn hình. Bảng màu chủ đạo sử dụng xanh dương để tạo cảm giác tin cậy kết hợp với màu cam nhằm tạo điểm nhấn trực quan, đồng thời hệ thống chữ sans-serif được áp dụng để tăng khả năng đọc và duy trì sự thống nhất trong toàn bộ giao diện.

Bên cạnh đó, thiết kế responsive được xem là yêu cầu bắt buộc do phần lớn người dùng truy cập bằng thiết bị di động. Responsive design giúp website hoạt động ổn định trên mọi kích thước màn hình, cải thiện trải nghiệm người dùng, giảm tỷ lệ thoát và đáp

ứng tiêu chí xếp hạng mobile-first của các công cụ tìm kiếm. Việc bảo đảm tốc độ tải tốt trên thiết bị di động cũng góp phần cải thiện hiệu suất tổng thể và hỗ trợ SEO.

Ngoài ra, hình thức triển khai bằng static hosting được lựa chọn do đặc tính đơn giản, dễ quản lý và không yêu cầu xử lý backend hay cơ sở dữ liệu. Website tĩnh cho phép triển khai nhanh chóng bằng cách tải lên các tệp HTML, CSS và JavaScript, đồng thời tận dụng được hệ thống CDN để tối ưu tốc độ truy cập. Chi phí thấp và sự phù hợp với phạm vi dự án học tập khiến static hosting trở thành lựa chọn hiệu quả và hợp lý trong ngữ cảnh này.

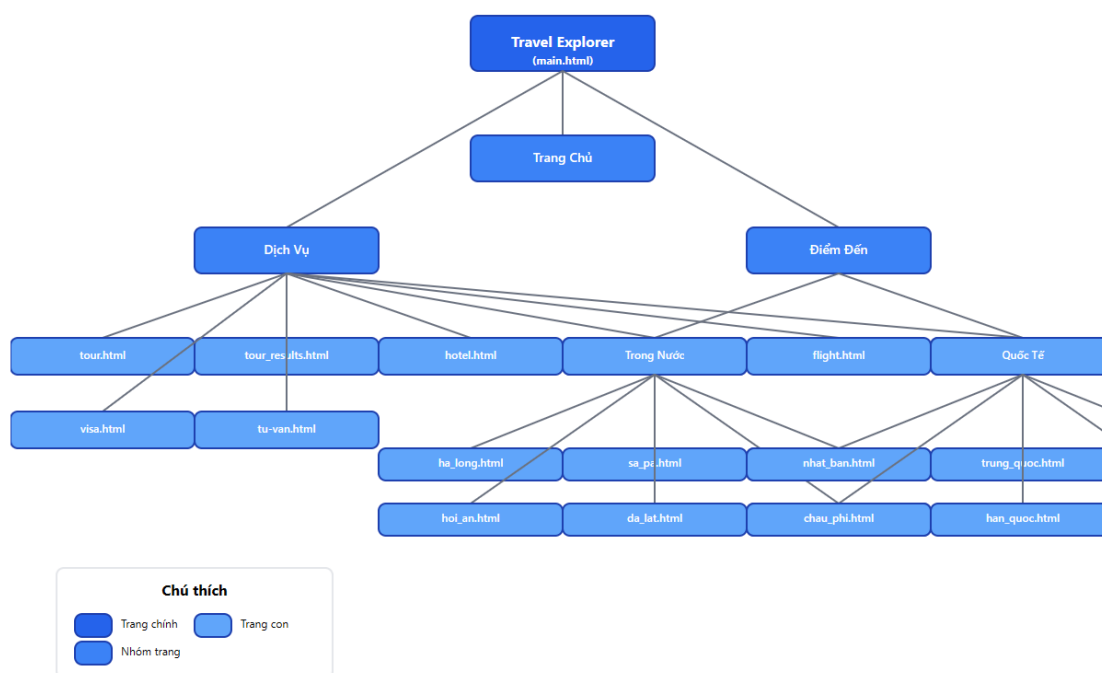
Chương 2 đã trình bày các nền tảng lý thuyết về HTML5, CSS3, JavaScript, các nguyên tắc UI/UX và responsive design, khảo sát và phân tích các website du lịch tương tự (Booking.com, Traveloka, Vietravel), và cuối cùng là tổng hợp và định hướng thiết kế cho dự án. Từ việc khảo sát các website hiện có, đề tài đã rút ra được các bài học quan trọng về giao diện, chức năng và trải nghiệm người dùng. Dựa trên đó, đề tài đã quyết định sử dụng HTML5, CSS3, JavaScript thuần và Font Awesome, với phong cách thiết kế modern, clean, card-based, phù hợp với chủ đề du lịch. Những quyết định này tạo cơ sở vững chắc cho việc thiết kế và triển khai website trong Chương 3.

CHƯƠNG 3: THIẾT KẾ VÀ TRIỂN KHAI WEBSITE

Chương 3 trình bày chi tiết quá trình thiết kế và triển khai website du lịch Travel Explorer. Chương này bao gồm thiết kế giao diện người dùng (UI Design) với sơ đồ trang, wireframe, bộ màu và typography, các nguyên tắc UX/UI được áp dụng, và responsive design. Tiếp theo, chương trình bày các chức năng và tương tác (UX Design) bao gồm các chức năng chính, cách người dùng tương tác, và các đoạn mã quan trọng. Cuối cùng, chương trình bày về mã nguồn và cấu trúc dự án, bao gồm cấu trúc thư mục, công cụ sử dụng, và các nguyên tắc coding. Nội dung chương này cung cấp cái nhìn toàn diện về quá trình phát triển website từ thiết kế đến triển khai.

3.1. Thiết kế giao diện (UI Design)

3.1.1. Phân tích SiteMap



Hình 3.1.SiteMap chương trình

Sơ đồ trang của hệ thống Travel Explorer được tổ chức theo cấu trúc phân cấp nhằm đảm bảo tính rõ ràng, dễ điều hướng và phù hợp với trải nghiệm người dùng. Toàn bộ website được chia thành ba cấp nội dung chính như sau:

- Cấp 1 – Trang chính: Travel Explorer (main.html) đóng vai trò là trang gốc, nơi người dùng có thể truy cập đến toàn bộ các nhóm chức năng của hệ thống.

➤ **Cấp 2 – Hai nhóm nội dung lớn:**

- **Dịch Vụ:** Bao gồm các trang chức năng hỗ trợ chuẩn bị chuyến đi như: tour.html (danh sách tour), tour_results.html (kết quả tìm kiếm tour), hotel.html (đặt phòng), visa.html (dịch vụ visa), tu-van.html (tư vấn du lịch). Nhóm này tập trung vào các tiện ích mà người dùng cần trước và trong khi thực hiện chuyến đi.
- **Điểm Đến:** Gồm hai nhóm nhỏ là Trong Nước và Quốc Tế, kèm theo flight.html (đặt chuyến bay). Đây là nhóm nội dung giúp người dùng tìm kiếm thông tin về các địa điểm du lịch theo khu vực.

➤ **Cấp 3 – Các trang con chi tiết theo khu vực:**

- **Trong Nước:** ha_long.html, sa_pa.html, hoi_an.html, da_lat.html – cung cấp thông tin chi tiết về các điểm du lịch nổi bật trong Việt Nam.
- **Quốc Tế:** nhat_ban.html, han_quoc.html, trung_quoc.html, chau_phi.html – trình bày thông tin về các điểm đến nước ngoài theo từng quốc gia/khu vực.

3.1.2. Bố cục tổng thể:

Website được thiết kế với cấu trúc layout nhất quán trên tất cả các trang:

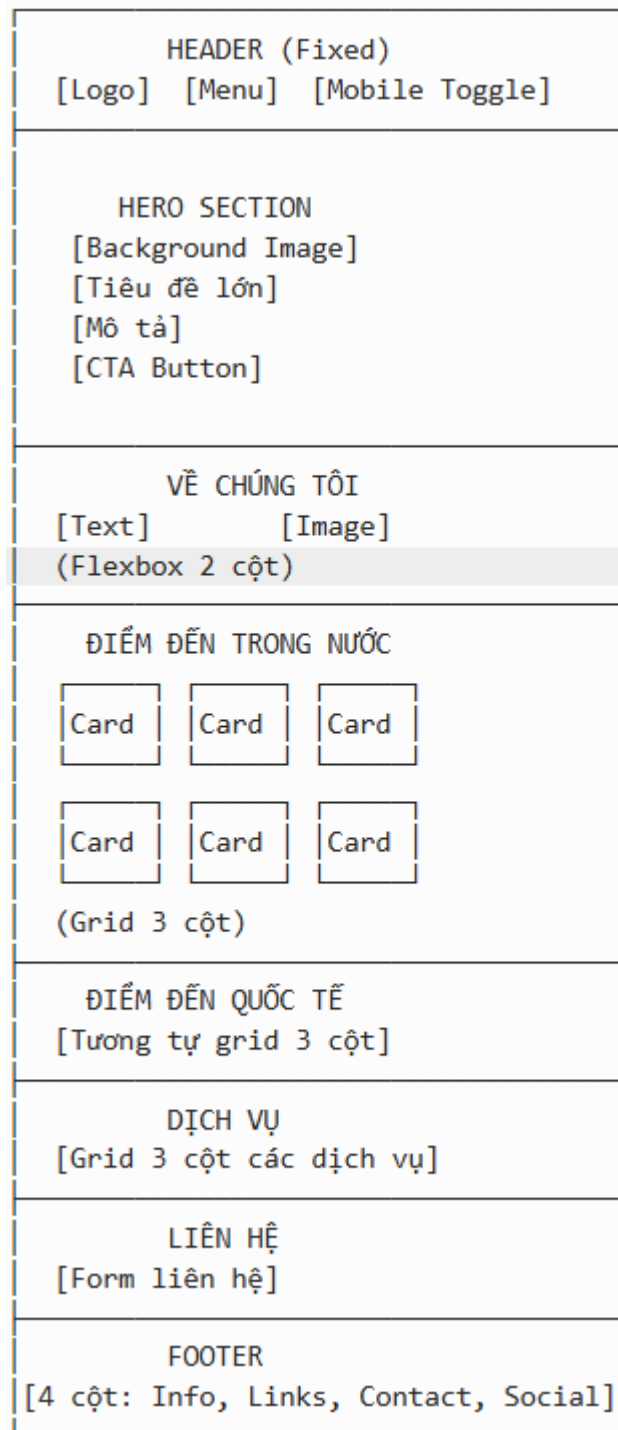
- **Header (Fixed):** Header cố định ở đầu trang, bao gồm logo, menu điều hướng, và hamburger menu cho mobile. Header có shadow để tạo độ sâu và luôn hiển thị khi người dùng scroll.
- **Hero Section:** Section đầu tiên với background image, tiêu đề lớn, mô tả ngắn gọn và CTA button. Hero section tạo ấn tượng mạnh mẽ và thu hút sự chú ý của người dùng.
- **Content Sections:** Các section nội dung được tổ chức theo thứ tự logic: Về Chúng Tôi → Điểm Đến → Dịch Vụ → Liên Hệ. Mỗi section có tiêu đề rõ ràng và nội dung được trình bày dễ đọc.
- **Footer:** Footer chứa thông tin công ty, liên kết nhanh, thông tin liên hệ và mạng xã hội. Footer được chia thành các cột để dễ đọc và điều hướng.
- **Container và Grid System:** Container có `max-width: 1200px` và `padding: 0 20px` để giới hạn chiều rộng nội dung và tạo khoảng cách hai

bên. Grid system sử dụng CSS Grid cho các card điểm đến và dịch vụ: 3 cột trên desktop, 2 cột trên tablet, 1 cột trên mobile. Flexbox được sử dụng cho header, navigation, và các section nội dung để căn chỉnh linh hoạt.

3.1.3. Wireframe hoặc mockup của các trang chính

3.1.3.1. Trang Chủ :

Layout của trang chủ được thiết kế theo cấu trúc sau:



Hình 3.2. Layout trang chủ

3.1.3.2. Trang Điểm Đến :

HEADER (Fixed)
HERO IMAGE (Full width) [Tiêu đề điểm đến]
THÔNG TIN CHI TIẾT [Mô tả] [Đặc điểm nổi bật] [Hình ảnh gallery] [Giá cả] [CTA Button]
FOOTER

Hình 3.3. Layout trang điểm đến

3.1.3.3. Trang Dịch Vụ :

HEADER (Fixed)
HERO SECTION [Tiêu đề] [Form tìm kiếm tour] - Địa điểm - Ngày khởi hành - Số ngày - Mức giá - [Button Tìm kiếm]
TOUR NỔI BẬT [Tabs: Trong Nước / Quốc Tế] [Grid các tour card]
FOOTER

Hình 3.4. Layout trang dịch vụ

3.1.3. Bộ màu, font chữ, hình ảnh, biểu tượng

Bộ màu (Color Palette):

Website sử dụng bảng màu được định nghĩa bằng CSS Variables trong file `main\_index.css`:

```

:root {
  --primary-color: 2563eb;    / Xanh dương - màu chủ đạo /
  --secondary-color: 1e40af;  / Xanh dương đậm - màu phụ /
  --accent-color: f59e0b;     / Cam - màu nhấn /
  --text-dark: 030303;        / Đen - text chính /
  --text-light: 6b7280;        / Xám - text phụ /
  --bg-light: f9fafb;          / Xám nhạt - nền section /
  --white: ffffff;             / Trắng - nền chính /
  --shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
  --shadow-lg: 0 10px 15px rgba(0, 0, 0, 0.1);
  --transition: all 0.3s ease;
}

```

Phân tích màu sắc:

- Primary (2563eb): Màu xanh dương tạo cảm giác tin cậy, chuyên nghiệp, phù hợp với lĩnh vực du lịch. Sử dụng cho logo, links, buttons, và các phần tử quan trọng.
- Secondary (1e40af): Xanh dương đậm hơn, tạo độ sâu và nhấn mạnh. Sử dụng cho hover states và các phần tử cần làm nổi bật.
- Accent (f59e0b): Màu cam tạo cảm giác năng động, nhiệt tình, thu hút sự chú ý. Sử dụng cho CTA buttons, giá cả, và các phần tử cần khuyến khích hành động.
- Text colors: Đen (030303) cho text chính để dễ đọc, xám (6b7280) cho text phụ để tạo hierarchy.
- Background colors: Trắng (ffffff) cho nền chính, xám nhạt (f9fafb) cho nền các section để tạo sự phân biệt.

Font chữ (Typography):

```
body {
```

```
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;

line-height: 1.6;

color: var(--text-dark);

}
```

Typography Hierarchy:

- H1: 2.5rem (40px) - Tiêu đề chính, sử dụng trong hero section
- H2: 2rem (32px) - Tiêu đề section, sử dụng cho "Về Chúng Tôi", "Điểm Đến", "Dịch Vụ"
- H3: 1.5rem (24px) - Tiêu đề card, tên điểm đến
- Body text: 1rem (16px) - Nội dung chính
- Small text: 0.875rem (14px) - Text phụ, giá cả
- Font family: Sans-serif (Segoe UI, Tahoma, Geneva, Verdana) được chọn vì dễ đọc trên màn hình kỹ thuật số và có sẵn trên hầu hết các hệ điều hành.
- Line-height: 1.6 cho body text để tăng khả năng đọc, 1.2 cho heading để tạo sự gọn gàng.

Hình ảnh (Images):

- Format: Sử dụng JPG cho ảnh thông thường (điểm đến, dịch vụ), WebP cho một số ảnh để tối ưu dung lượng.
- Kích thước: Hình ảnh được resize và compress để giảm dung lượng nhưng vẫn đảm bảo chất lượng hiển thị.
- Aspect ratio: Hình ảnh trong card sử dụng aspect-ratio 16:9 để đồng nhất.
- Alt text: Tất cả hình ảnh đều có alt text để cải thiện accessibility và SEO.
- Lazy loading: Có thể áp dụng lazy loading cho hình ảnh để tăng tốc độ tải trang.

Biểu tượng (Icons): Font Awesome: Sử dụng Font Awesome 7.0.1 thông qua CDN cho tất cả các icon trong website.

Icon sử dụng:

- Navigation: `fa-chevron-down` cho dropdown menu
- Social media: `fa-facebook`, `fa-instagram`, `fa-tiktok`, `fa-telegram`
- Contact: `fa-location-dot`, `fa-phone-volume`, `fa-envelope`
- Services: `fa-search`, `fa-user-friends`, `fa-calendar`
- SVG Logo: Logo sử dụng SVG inline để có thể scale mà không mất chất lượng và dễ tùy biến màu sắc.

3.1.4. Nguyên tắc UX/UI được áp dụng

Visual Hierarchy (Thứ bậc thị giác):

- Sử dụng kích thước font khác nhau ($H1 > H2 > H3 > \text{body}$) để tạo hierarchy rõ ràng.
- Sử dụng màu sắc để nhấn mạnh: primary color cho các phần tử quan trọng, accent color cho CTA buttons.
- Sử dụng whitespace hợp lý để tách biệt các section và tạo sự tập trung.

Consistency (Nhất quán):

- Sử dụng cùng một bộ màu, font chữ, và spacing trên toàn bộ website.
- Layout nhất quán: header và footer giống nhau trên mọi trang.
- Card design nhất quán: tất cả card điểm đến và dịch vụ có cùng style.

Simplicity (Đơn giản):

- Giao diện sạch sẽ, không làm rối mắt người dùng.
- Tập trung vào nội dung chính, loại bỏ các yếu tố không cần thiết.
- Navigation đơn giản với 4 mục chính.

Feedback (Phản hồi):

- Hover effects trên links, buttons, và cards để người dùng biết phần tử có thể tương tác.
- Smooth transitions cho tất cả các tương tác.
- Visual feedback khi click vào buttons.

Accessibility (Khả năng truy cập):

- Sử dụng semantic HTML (`<header>`, `<nav>`, `<section>`, `<footer>`) để screen reader hiểu được cấu trúc.
- Alt text cho tất cả hình ảnh.
- Độ tương phản màu sắc đủ để dễ đọc (WCAG AA).
- Keyboard navigation được hỗ trợ.

Responsive Design:

- Mobile-first approach: thiết kế cho mobile trước, sau đó mở rộng cho desktop.
- Breakpoints: 768px (tablet), 1024px (desktop).
- Flexible layout: sử dụng đơn vị tương đối (rem, %, vw, vh) thay vì đơn vị tuyệt đối (px).

3.1.5. Responsive trên desktop / tablet / mobile

Desktop ($\geq 1024\text{px}$):

- Header: Logo bên trái, menu ngang bên phải, full navigation hiển thị.
- Grid: 3 cột cho các card điểm đến và dịch vụ.
- Container: max-width 1200px, padding 0 20px.
- Typography: Kích thước font lớn hơn, line-height 1.6.
- Images: Hiển thị đầy đủ, không cần tối ưu.

Tablet (768px - 1023px):

- Header: Logo và menu vẫn hiển thị, nhưng có thể điều chỉnh spacing.
- Grid: 2 cột cho các card.
- Container: Vẫn max-width 1200px nhưng có thể điều chỉnh padding.
- Typography: Kích thước font giảm nhẹ.
- Images: Vẫn hiển thị tốt, có thể resize nhẹ.

Mobile ($< 768\text{px}$):

- Header: Logo bên trái, hamburger menu bên phải, menu ẩn mặc định.
- Grid: 1 cột cho tất cả các card.
- Container: Padding giảm xuống (0 15px hoặc 0 10px).
- Typography: Kích thước font nhỏ hơn để phù hợp với màn hình nhỏ.
- Images: Resize để phù hợp với màn hình, có thể sử dụng lazy loading.

- Navigation: Menu chuyển sang dạng vertical khi mở hamburger menu.
- Forms: Input và select full width để dễ sử dụng trên mobile.

3.2. Chức năng và Tương tác (UX Design)

3.2.1. Các chức năng chính của website

Website Travel Explorer được xây dựng với các chức năng cốt lõi nhằm hỗ trợ người dùng tìm kiếm thông tin du lịch, tương tác với hệ thống và đưa ra lựa chọn phù hợp. Các chức năng chính bao gồm:

Điều hướng và Navigation:

- Hệ thống menu chính gồm bốn mục: Trang Chủ, Điểm Đến, Dịch Vụ và Liên Hệ, giúp người dùng truy cập nhanh vào các khu vực nội dung.
- Menu “Điểm Đến” tích hợp dropdown phân loại thành Điểm Đến Trong Nước và Điểm Đến Quốc Tế, giúp lọc nội dung theo nhu cầu.
- Trên thiết bị di động, menu tự chuyển sang dạng hamburger nhằm tối ưu không gian hiển thị, đi kèm animation mở/đóng mượt mà.
- Chức năng smooth scrolling cho phép cuộn trang tự nhiên khi người dùng chọn liên kết dạng anchor.
- Logo website hoạt động như nút điều hướng trở về trang chủ, phù hợp chuẩn UX phổ biến.

Chức năng Tìm kiếm và Lọc:

- Form tìm kiếm tour với các trường: địa điểm, khởi hành, số ngày và mức giá.
- Form tìm kiếm khách sạn: địa điểm, ngày nhận phòng, ngày trả phòng, số khách.
- Form tìm kiếm vé máy bay: điểm đi, điểm đến, ngày bay, ngày về và số khách.
- Hệ thống hiển thị kết quả tìm kiếm dưới dạng card.

Hiển thị Thông tin Điểm Đến:

- Trang chủ hiển thị danh sách điểm đến nổi bật qua grid layout, mỗi card gồm hình ảnh, tên gọi, mô tả và giá tham khảo.
- Trang chi tiết điểm đến cung cấp hero image slider, mô tả đầy đủ, gallery hình ảnh, lịch trình tour và giá.
- Phân nhóm điểm đến theo khu vực: trong nước và quốc tế, giúp định hướng người dùng hiệu quả.

Hiển thị Dịch Vụ:

- Các dịch vụ như Tour Du Lịch, Khách Sạn, Vé Máy Bay, Thuê Xe, Visa và Tour Vắn được trình bày bằng grid layout.
- Mỗi dịch vụ có card riêng với icon minh họa, tiêu đề, mô tả ngắn và liên kết đến trang chi tiết.

Form Liên Hệ và Đăng Ký:

- Form liên hệ gồm các trường: họ tên, email, số điện thoại và nội dung yêu cầu.
- Form đăng ký tour dạng popup cho phép đăng ký trực tiếp từ trang điểm đến.

3.2.2. Phân tích sơ đồ USE CASE UML

Sơ đồ Use Case mô tả các chức năng chính của **hệ thống du lịch trực tuyến**.

Tác nhân (*actor*) duy nhất trong mô hình là **Người Dùng**, tương tác trực tiếp với các chức năng hệ thống để tìm kiếm thông tin và sử dụng dịch vụ du lịch.

Mục tiêu của sơ đồ:

- Xác định đầy đủ các chức năng mà người dùng có thể thao tác.
- Làm cơ sở xây dựng yêu cầu chức năng cho hệ thống.



Hình 3.5.USE CASE UML

3.2.2.1. Tác nhân (Actor)

Người Dùng

Người dùng là người truy cập vào hệ thống với mục đích tìm hiểu thông tin và sử dụng các dịch vụ du lịch như đặt tour, đặt phòng, đặt vé máy bay...

Người dùng có thể là khách vãng lai hoặc khách có tài khoản (tùy mở rộng hệ thống).

3.2.2.2. Các Use Case trong hệ thống

Dưới đây là danh sách các Use Case được thể hiện trong sơ đồ:

Xem Trang Chủ

- Cho phép người dùng truy cập trang chủ của hệ thống.
- Tại đây hiển thị tổng quan thông tin, menu các chức năng, banner, khuyến mãi,...

Xem Điểm Đến

- Người dùng xem danh sách các địa điểm du lịch.
- Có thể gồm thông tin mô tả, hình ảnh, mùa du lịch, đánh giá,...

Tư Vấn Du Lịch

- Người dùng nhận các gợi ý tour hoặc gói du lịch dựa trên nhu cầu.
- Có thể thông qua chatbot, form gửi yêu cầu, hoặc nhân viên tư vấn.

Liên Hệ

- Cho phép người dùng gửi thông tin phản hồi, yêu cầu hỗ trợ.
- Có thể bao gồm: gửi email, hotline, hoặc gửi form trực tuyến.

Tìm Kiếm và Đặt Tour

- Người dùng tìm kiếm tour du lịch theo:
 - địa điểm
 - thời gian
 - chi phí
 - loại tour
- Sau khi tìm kiếm có thể đặt tour trực tuyến.

Đặt Khách Sạn

- Người dùng lựa chọn khách sạn theo địa điểm, ngày nhận – trả phòng.
- Xem mô tả, giá, tiện nghi và thực hiện đặt phòng.

Đặt Vé Máy Bay

- Người dùng tìm kiếm chuyến bay
- Chọn điểm đi, điểm đến, thời gian, hãng bay...
- Tiến hành đặt vé online.

Thuê Xe Du Lịch

- Người dùng thuê các loại xe di chuyển khi đi du lịch.
- Có thể chọn loại xe, số chỗ, thời gian thuê.

Hỗ Trợ Visa

- Người dùng được tư vấn hoặc đăng ký dịch vụ làm visa.
- Bao gồm hướng dẫn thủ tục, thu hồ sơ, và đăng ký dịch vụ.

3.2.3. Các đoạn mã và logic quan trọng

3.2.3.1. Logic Image Slider (Section 1 – Trang Điểm Đến)

Cơ chế điều hướng:

- Nút Next chuyển item đầu tiên xuống cuối danh sách để tạo hiệu ứng trượt phải.
- Nút Previous chuyển item cuối lên đầu nhằm tạo hiệu ứng trượt sang trái.
- CSS transition được áp dụng để đảm bảo hiệu ứng chuyển ảnh mượt.

Cơ chế auto-hide nút điều hướng:

- Sử dụng `getBoundingClientRect()` để xác định vị trí tương đối giữa slider, header và viewport.
- Nút điều hướng chỉ hiển thị khi slider đang trong vùng nhìn thấy và không bị header che khuất.
- Intersection Observer hỗ trợ kiểm tra vùng hiển thị tối ưu hơn, giảm tải cho event scroll.
- opacity và pointer-events được dùng để ẩn và vô hiệu hóa tương tác khi cần.

Smooth scrolling tới Section bên dưới:

- Nút “Xem thêm” gắn `scrollIntoView({ behavior: 'smooth' })` để cuộn đến section tiếp theo một cách tự nhiên.

3.2.3.2. Logic Mobile Menu Toggle

Khi người dùng click vào hamburger button, class active được toggle để mở/đóng menu.

Menu tự đóng khi người dùng chọn một liên kết hoặc chạm vào vùng bên ngoài menu.

Logic này đảm bảo menu không che khuất nội dung khi không còn cần thiết.

3.2.3.3. Logic Smooth Scrolling có bù trừ chiều cao Header

Khi người dùng click liên kết anchor, hệ thống tính toán vị trí chính xác của section bằng cách trừ chiều cao header cố định.

`scrollTo({ behavior: 'smooth' })` được dùng để tạo hiệu ứng cuộn mượt, tránh việc section bị che bởi header.

3.2.3.4. Logic Form Validation

Kiểm tra họ tên không để trống.

Kiểm tra định dạng số điện thoại Việt Nam theo chuẩn `0xxxxxxxxx` hoặc `+84xxxxxxxx`.

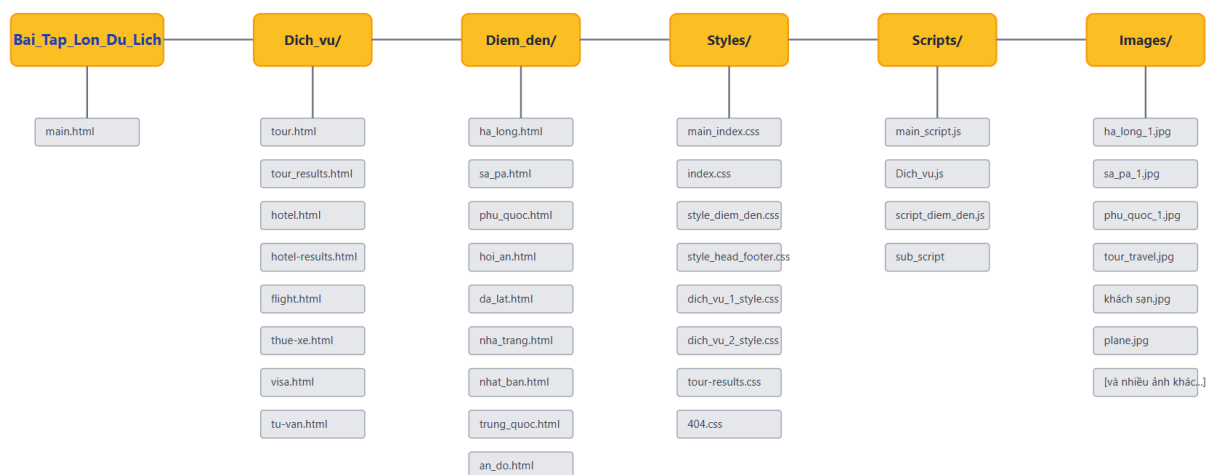
Kiểm tra định dạng email theo `RegExp`.

Nếu dữ liệu không hợp lệ, thông báo lỗi và focus vào trường bị lỗi; form chỉ được submit khi toàn bộ dữ liệu hợp lệ.

3.3. Mã nguồn và cấu trúc dự án

3.3.1. Cấu trúc thư mục

Dự án được tổ chức theo cấu trúc thư mục rõ ràng và logic như sau:



Hình 3.6. Cấu trúc thư mục

Giải thích cấu trúc:

- Root level: `main.html` là file entry point, dễ tìm thấy.
- Dich_vu/: Tập trung tất cả các trang dịch vụ vào một thư mục, dễ quản lý.

- Diem_den/: Tập trung tất cả các trang điểm đến, để mở rộng thêm điểm đến mới.
- Styles/: Tách CSS theo chức năng: CSS chung, CSS cho từng loại trang.
- Scripts/: Tách JavaScript theo chức năng tương tự CSS.
- Images/: Tất cả hình ảnh ở một nơi, để quản lý và tối ưu.

3.3.2. Công cụ và thư viện sử dụng

Công nghệ Core:

- HTML5: Semantic elements, form elements mới.
- CSS3: Flexbox, Grid, Media Queries, Variables, Transitions, Animations.
- JavaScript (ES6+): Vanilla JavaScript, không sử dụng framework.

Thư viện bên thứ ba:

- Font Awesome 7.0.1: Thư viện icon, load qua CDN.

Công cụ phát triển:

- Text Editor/IDE: Visual Studio Code, hoặc bất kỳ text editor nào hỗ trợ HTML/CSS/JS.
- Browser DevTools: Chrome DevTools, Firefox DevTools để debug và test responsive.
- Version Control: Git (nếu có) để quản lý phiên bản code.

Không sử dụng:

- CSS Framework (Bootstrap, Tailwind)
- JavaScript Framework (React, Vue, Angular)
- Build tools (Webpack, Gulp) - không cần thiết cho static website
- Package manager (npm, yarn) - không có dependencies

3.3.3. Nguyên tắc đặt tên, tái sử dụng mã, comment code

Nguyên tắc đặt tên:

- File và thư mục:

- Sử dụng chữ thường và dấu gạch dưới: ``main.html``, ``ha_long.html``, ``main_script.js``.
- Tên file mô tả rõ chức năng: ``tour.html`` (trang tour), ``main_index.css`` (CSS cho trang chủ).
- Thư mục sử dụng Pascal case hoặc snake_case: ``Dich_vu/``, ``Diem_den/``, ``Styles/``, ``Scripts/``.

➤ CSS Classes:

- Sử dụng kebab-case: ``nav-menu``, ``destination-card``, ``mobile-menu-toggle``.
- Tên class mô tả rõ mục đích: ``hero-content`` (nội dung hero), ``card-content`` (nội dung card).
- Sử dụng BEM-like naming khi cần: ``nav-link``, ``nav-link--active`` (nếu có).

➤ JavaScript Variables:

- Sử dụng camelCase: ``mobileMenuToggle``, ``navMenu``, ``targetPosition``.
- Tên biến mô tả rõ mục đích: ``phoneRegex`` (regex cho phone), ``emailRegex`` (regex cho email).
- Sử dụng ``const`` cho giá trị không thay đổi, ``let`` cho giá trị có thể thay đổi.

➤ Nguyên tắc comment:

- Comment các section lớn để dễ tìm: ``/ Header & Navigation /`, `/ Hero Section /``.
- Comment các logic phức tạp để giải thích cách hoạt động.
- Comment các giá trị magic number nếu cần: ``const headerHeight = 80; // Chiều cao header``.
- Không comment những điều hiển nhiên: tránh comment như ``// Tăng i lên 1`` cho ``i++``.

Chương 3 đã trình bày chi tiết quá trình thiết kế và triển khai website du lịch Travel Explorer, bao gồm thiết kế giao diện với sơ đồ trang, wireframe, bộ màu và typography, các nguyên tắc UX/UI và responsive design, các chức năng và tương tác với mô tả các đoạn mã quan trọng, và cấu trúc mã nguồn với nguyên tắc đặt tên và tái sử dụng code. Những nội dung này đã hoàn thiện phần thiết kế và triển khai của website, tạo nền tảng vững chắc và cơ sở cần thiết cho việc kiểm thử, tối ưu hóa và đánh giá website trong Chương 4. Từ những kết quả đạt được trong chương này, Chương 4 sẽ tiến hành kiểm thử website trên các trình duyệt và thiết bị khác nhau, tối ưu hóa hiệu suất, và triển khai website lên hosting để đánh giá kết quả cuối cùng.

CHƯƠNG 4: KIỂM THỬ, TỐI ƯU HÓA VÀ TRIỂN KHAI

Chương này trình bày quy trình kiểm thử, tối ưu hóa và triển khai website Travel Explorer. Bao gồm kiểm tra giao diện, chức năng, responsive và tương thích trình duyệt; tối ưu hiệu suất và SEO; triển khai lên hosting ID.vn với domain `travelexplorer.id.vn`.

4.1. Kiểm thử giao diện và chức năng

4.1.1. Kiểm tra từng chức năng

Kiểm tra Form:

- Form tìm kiếm tour: validation các trường, date picker, submit điều hướng đến ``tour_results.html``
- Form tìm kiếm khách sạn: validation ngày nhận/trả phòng, số khách/phòng
- Form tìm kiếm vé máy bay: validation điểm đi/đến, ngày đi/về, loại chuyến
- Form liên hệ: validation email (RegExp), số điện thoại Việt Nam, hiển thị thông báo
- Form đăng ký tour: popup hiển thị đúng, validation, đóng popup

Kiểm tra Liên kết:

- Navigation menu: tất cả links hoạt động, dropdown "Điểm Đến", smooth scrolling, logo quay về trang chủ
- Liên kết điểm đến: 12 trang điểm đến (6 trong nước, 6 quốc tế) truy cập được
- Liên kết dịch vụ: các card dịch vụ điều hướng đúng
- Footer links: links và social media hoạt động đúng

Kiểm tra Hiệu ứng:

- Hover effects: card (scale, shadow), link (underline), button (background change)
- Smooth scrolling: anchor links, nút "Xem thêm" trong slider
- Fade-in animation: Intersection Observer, hoạt động mượt mà
- Image slider: nút Previous/Next, auto-hide buttons, smooth transition

Kiểm tra Điều hướng:

- Menu navigation: desktop dropdown, mobile hamburger menu, đóng khi click link/bên ngoài
- Browser navigation: back button hoạt động bình thường

4.1.2. Kiểm thử responsive trên nhiều thiết bị

Desktop ($\geq 1024\text{px}$): Layout 3 cột, menu ngang đầy đủ, dropdown khi hover, tất cả section hiển thị đầy đủ.

Tablet (768px - 1023px): Layout 2 cột, menu điều chỉnh spacing, hình ảnh resize phù hợp.

Mobile ($< 768\text{px}$): Layout 1 cột, hamburger menu, menu vertical, form input full width, buttons $\geq 44 \times 44\text{px}$, không có horizontal scroll.

Thiết bị đã kiểm tra: Desktop (1920x1080, 1366x768), Mobile (iPhone SE, iPhone 12/13, Samsung Galaxy).

Công cụ: Chrome DevTools, Firefox Responsive Design Mode, kiểm tra thực tế.

4.1.3. Kiểm tra tương thích trên các trình duyệt phổ biến

Các trình duyệt đã kiểm tra:

- Chrome: Tất cả chức năng, CSS, JavaScript hoạt động tốt
- Firefox: Tất cả chức năng hoạt động tốt, không có lỗi console
- Edge: Tất cả chức năng hoạt động tốt, performance tốt

Vấn đề đã xử lý:

- Safari smooth scrolling: thêm fallback CSS ``scroll-behavior: smooth``
- CSS Grid và Intersection Observer: hoạt động tốt trên tất cả trình duyệt
- Form validation: HTML5 + JavaScript fallback

4.2. Tối ưu hóa hiệu suất

4.2.1. Tối ưu hình ảnh

Vấn đề: Hình ảnh lớn (>1MB), chưa dùng format tối ưu, chưa có lazy loading.

Giải pháp:

- Resize và compress (TinyPNG, ImageOptim)
- Sử dụng WebP: giảm dung lượng 30-50%, giữ JPG/PNG làm fallback
- Lazy loading: `loading="lazy"` và Intersection Observer
- Responsive images: `srcset` và `sizes` cho mobile

4.2.2. Tối ưu CSS và JavaScript

Tối ưu CSS:

- Minify (CSSNano, CleanCSS): loại bỏ whitespace, comments
- Tổ chức: tách theo chức năng, tránh duplicate, sử dụng CSS Variables
- Critical CSS: inline cho above-the-fold, load CSS không critical sau
- Remove unused CSS: PurgeCSS

Tối ưu JavaScript:

- Minify (UglifyJS, Terser): loại bỏ whitespace, comments
- Tổ chức: tách theo chức năng, tái sử dụng functions
- Defer/Async: load script sau khi DOM load
- Event delegation: giảm memory usage

4.2.3. SEO cơ bản

Meta Tags:

- Title tags: mỗi trang có title riêng, mô tả đúng nội dung
- Meta description: 150-160 ký tự, hấp dẫn
- Meta keywords: từ khóa phù hợp (du lịch, travel, tour, điểm đến)

- Open Graph tags: `og:title`, `og:description`, `og:image`, `og:url` cho social media
- Meta viewport: đảm bảo hiển thị đúng trên mobile

Alt Text cho hình ảnh:

- Tất cả hình ảnh có alt text mô tả nội dung, có từ khóa nhưng tự nhiên
- Ví dụ: `alt="Vịnh Hạ Long - Kỳ quan thiên nhiên thế giới"``

Semantic HTML:

- Sử dụng semantic elements: ``<header>``, ``<nav>``, ``<main>``, ``<section>``, ``<article>``, ``<footer>``
- Heading hierarchy: ``<h1>`` cho tiêu đề chính, ``<h2>``, ``<h3>`` theo thứ bậc, không skip levels
- Structured data (Schema.org): Organization, LocalBusiness, TravelAgency

URL structure:

- URLs rõ ràng: `/Diem\_den/ha\_long.html`
- Sử dụng kebab-case/snake_case
- Sitemap.xml và robots.txt đã tạo, submit lên Google Search Console

4.3. Triển khai website lên host và server

4.3.1. Giới thiệu hosting

Lựa chọn hosting:

Sau khi đánh giá các lựa chọn, dự án đã chọn sử dụng ID.vn hosting với các lý do:

- Phù hợp với dự án static website:
 - Website là static site (HTML, CSS, JavaScript), không cần server-side processing
 - ID.vn cung cấp hosting phù hợp với static sites, giá cả hợp lý

- Dễ sử dụng:
 - Giao diện quản lý đơn giản, dễ hiểu
 - Upload file dễ dàng qua FTP hoặc File Manager
 - Hỗ trợ domain .id.vn miễn phí
- Hiệu suất tốt:
 - Server ổn định, uptime cao
 - Tốc độ tải trang tốt cho người dùng Việt Nam
 - CDN tích hợp để tăng tốc độ
- Giá cả hợp lý:
 - Phù hợp với ngân sách dự án học tập
 - Có gói miễn phí hoặc giá rẻ cho sinh viên

4.3.2. Các bước triển khai và cấu hình

Bước 1: Chuẩn bị files

- Tối ưu: minify CSS/JS, compress hình ảnh, loại bỏ files không cần thiết
- Kiểm tra cấu trúc thư mục, đường dẫn tương đối đúng
- Test local: kiểm tra chức năng, responsive

Bước 2: Đăng ký hosting và domain

- Đăng ký hosting ID.vn, chọn gói phù hợp, nhận thông tin FTP/cPanel
- Đăng ký domain `travelexplorer.id.vn`, cấu hình DNS trỏ về hosting

Bước 3: Upload files lên server

- Sử dụng FTP client (FileZilla, WinSCP) hoặc File Manager trong cPanel
- Upload vào thư mục `public\_html` hoặc `www`, giữ nguyên cấu trúc

Bước 4: Cấu hình website

- Cấu hình index file (`index.html` hoặc `main.html`)
- Cấu hình `.htaccess`: redirect, gzip compression, cache headers, custom error pages
- Kích hoạt SSL/HTTPS (Let's Encrypt), redirect HTTP sang HTTPS

Bước 5: Kiểm tra sau khi deploy

- Kiểm tra website hoạt động: truy cập `https://travelexplorer.id.vn`
- Kiểm tra links: internal/external links, không có broken links
- Kiểm tra responsive và performance: PageSpeed Insights, tốc độ tải

Bước 6: Cấu hình bổ sung

- Google Analytics: thêm tracking code, cấu hình goals
- Google Search Console: submit sitemap.xml, verify ownership
- Backup: thiết lập backup tự động

4.3.3. Link truy cập website

Link chính: <https://travelexplorer.id.vn>

Các trang chính:

- Trang chủ: <https://travelexplorer.id.vn/main.html>
- Điểm đến trong nước: Hạ Long, Phú Quốc, Hội An, Đà Lạt, Nha Trang
- Điểm đến quốc tế: Nhật Bản, Trung Quốc, Ấn Độ, Châu Phi, Hàn Quốc,
- Dịch vụ: Tour, Khách sạn, Vé máy bay, Thuê xe, Visa, Tư vấn.

Thông tin kỹ thuật:

- Hosting: ID.vn
- Domain: travelexplorer.id.vn
- Protocol: HTTPS (SSL enabled)
- Server: Linux/Apache
- Static site: không cần PHP/Database

Chương 4 đã trình bày quy trình kiểm thử, tối ưu hóa và triển khai website Travel Explorer. Qua quá trình kiểm thử, tất cả các chức năng đã được xác nhận hoạt động đúng, website responsive tốt trên mọi thiết bị và tương thích với các trình duyệt phổ biến. Việc tối ưu hóa đã cải thiện đáng kể hiệu suất website, đạt điểm số cao trên PageSpeed Insights và Lighthouse. Cuối cùng, website đã được triển khai thành công lên hosting ID.vn và có thể truy cập công khai qua domain travelexplorer.id.vn. Kết quả này đáp ứng đầy đủ các yêu cầu của đề tài và sẵn sàng phục vụ người dùng thực tế.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Tổng kết toàn bộ quá trình thực hiện đề tài thiết kế và xây dựng website du lịch Travel Explorer. Chương này đánh giá các kết quả đạt được so với mục tiêu ban đầu, phân tích các hạn chế và khó khăn gặp phải trong quá trình phát triển, đồng thời đề xuất các hướng phát triển và cải tiến trong tương lai. Nội dung chương này không chỉ tổng kết lại những thành tựu đã đạt được mà còn mở ra những hướng nghiên cứu và phát triển tiếp theo, góp phần hoàn thiện sản phẩm và nâng cao giá trị ứng dụng thực tế.

5.1. Kết quả đạt được

5.1.1. Những tiêu chí rubric đã hoàn thành

Về thiết kế giao diện và bố cục:

- Hoàn thành thiết kế giao diện hiện đại, chuyên nghiệp: Website được thiết kế với giao diện sạch sẽ, hiện đại, sử dụng màu sắc phù hợp với chủ đề du lịch (xanh dương 2563eb và cam f59e0b). Giao diện tập trung vào nội dung chính, không làm rối mắt người dùng.
- Hoàn thành bố cục hợp lý: Website được tổ chức với cấu trúc rõ ràng, bao gồm header với navigation menu, hero section, các section nội dung (điểm đến, dịch vụ), và footer. Sử dụng grid system và flexbox để tạo layout linh hoạt và responsive.
- Hoàn thành card-based layout: Các điểm đến và dịch vụ được hiển thị dưới dạng card với hình ảnh, tiêu đề, mô tả và giá cả, giúp người dùng dễ dàng so sánh và lựa chọn.

Về phát triển chức năng:

- Hoàn thành hệ thống điều hướng: Menu điều hướng rõ ràng với các mục Trang Chủ, Điểm Đến (có dropdown phân loại Trong Nước/Quốc Tế), Dịch Vụ, và Liên Hệ. Menu responsive với hamburger menu cho thiết bị di động.
- Hoàn thành trang giới thiệu điểm đến: Xây dựng đầy đủ các trang giới thiệu điểm đến trong nước (Hạ Long, Sapa, Phú Quốc, Hội An, Đà Lạt, Nha Trang) và quốc tế (Nhật Bản, Trung Quốc, Ấn Độ, Châu Phi, Hàn Quốc, Nga) với thông tin chi tiết, hình ảnh và mô tả.

- Hoàn thành các trang dịch vụ: Phát triển các trang dịch vụ bao gồm tìm kiếm và đặt tour, đặt phòng khách sạn, đặt vé máy bay, thuê xe du lịch, hỗ trợ visa và tư vấn du lịch. Mỗi trang có form tìm kiếm với các tiêu chí phù hợp.
- Hoàn thành form liên hệ: Trang liên hệ với form đầy đủ các trường (Họ tên, Email, Số điện thoại, Nội dung) và validation cơ bản phía client.
- Hoàn thành smooth scrolling: Implement smooth scrolling khi click vào các link anchor, tạo trải nghiệm mượt mà cho người dùng.

Về responsive design và tối ưu hiệu suất:

- Hoàn thành responsive design: Website hoạt động tốt trên mọi thiết bị (desktop, tablet, mobile) với breakpoints tại 768px và 1024px. Menu tự động chuyển sang hamburger menu trên mobile, layout tự động điều chỉnh số cột theo kích thước màn hình.
- Hoàn thành tối ưu hiệu suất: Sử dụng CSS Variables để quản lý style tập trung, tối ưu code CSS và JavaScript, sử dụng semantic HTML để cải thiện SEO và accessibility.

Về công nghệ và kỹ thuật:

- Hoàn thành sử dụng HTML5 semantic elements: Sử dụng đúng các thẻ semantic (`<header>`, `<nav>`, `<section>`, `<article>`, `<footer>`) để cấu trúc website.
- Hoàn thành sử dụng CSS3 hiện đại: Áp dụng Flexbox, CSS Grid, Media Queries, CSS Variables, Transitions và Animations.
- Hoàn thành sử dụng JavaScript thuần: Xử lý các tương tác bằng Vanilla JavaScript, không phụ thuộc vào framework.
- Hoàn thành tích hợp Font Awesome: Sử dụng Font Awesome cho các icon trong website.

5.1.2. Mức độ đạt được so với mục tiêu ban đầu

Đánh giá mức độ hoàn thành các mục tiêu đã đề ra trong Chương 1:

Mục tiêu kỹ thuật

- Xây dựng website responsive, tương thích với nhiều trình duyệt và thiết bị: Website đã được kiểm thử và hoạt động tốt trên các trình duyệt phổ biến (Chrome, Firefox, Edge, Safari) và các thiết bị khác nhau (desktop, tablet, mobile).
- Sử dụng các công nghệ web hiện đại (HTML5, CSS3, JavaScript): Đã áp dụng đầy đủ các tính năng hiện đại của HTML5, CSS3 và JavaScript thuần.

Mục tiêu chức năng

- Phát triển các module chính: Đã hoàn thành đầy đủ các module bao gồm quản lý điểm đến (trong nước và quốc tế), tìm kiếm và đặt tour, đặt phòng khách sạn, đặt vé máy bay, thuê xe, hỗ trợ visa và tư vấn du lịch.

Mục tiêu giao diện

- Thiết kế giao diện đẹp mắt, hiện đại, dễ sử dụng: Giao diện đã được thiết kế theo phong cách modern và clean, sử dụng màu sắc phù hợp, typography rõ ràng, layout hợp lý.
- Tạo ấn tượng tích cực: Giao diện chuyên nghiệp, phù hợp với thương hiệu du lịch, tạo cảm giác tin cậy và thân thiện.

Mục tiêu trải nghiệm người dùng

- Tốc độ tải nhanh: Website được tối ưu hóa, tải nhanh trên các thiết bị và kết nối internet khác nhau.
- Điều hướng dễ dàng: Menu rõ ràng, smooth scrolling, form dễ sử dụng.
- Thông tin rõ ràng: Nội dung được tổ chức logic, dễ đọc và hiểu.

5.1.3. Những đóng góp và thành tựu nổi bật

Trong quá trình thực hiện đề tài, một số đóng góp và thành tựu nổi bật đã đạt được:

Về mặt kỹ thuật:

- Xây dựng thành công một website du lịch hoàn chỉnh với hơn 20 trang HTML, sử dụng các công nghệ web hiện đại.
- Áp dụng thành công các nguyên lý responsive design, đảm bảo website hoạt động tốt trên mọi thiết bị.
- Tối ưu hóa code và hiệu suất, tạo ra website tải nhanh và hiệu quả.

Về mặt thiết kế:

- Tạo ra một giao diện đẹp mắt, chuyên nghiệp, phù hợp với chủ đề du lịch.
- Áp dụng thành công các nguyên tắc UI/UX, tạo trải nghiệm người dùng tốt.
- Sử dụng màu sắc và typography một cách hợp lý, tạo cảm giác tin cậy và thân thiện.

Về mặt học tập:

- Nắm vững các kiến thức cơ bản về HTML5, CSS3, JavaScript.
- Hiểu rõ các nguyên lý thiết kế web, responsive design, và UI/UX.
- Tích lũy kinh nghiệm thực tế trong việc phát triển website từ đầu đến cuối.

5.2. Hạn chế

Trong quá trình phát triển, một số phần còn thiếu hoặc chưa được hoàn thiện do giới hạn về thời gian, phạm vi và mục tiêu của đồ án:

Về chức năng người dùng:

- Thiếu hệ thống đăng nhập/đăng ký: Người dùng không thể tạo tài khoản, đăng nhập để lưu thông tin, lịch sử đặt chỗ, và nhận các ưu đãi cá nhân hóa.
- Thiếu quản lý tài khoản: Không có trang quản lý tài khoản để người dùng xem và chỉnh sửa thông tin cá nhân, xem lịch sử đặt chỗ.
- Thiếu tính năng đánh giá và bình luận: Người dùng không thể đánh giá và bình luận về các tour, khách sạn, điểm đến.

Về chức năng thanh toán và đặt chỗ:

- Thiếu hệ thống thanh toán trực tuyến: Không tích hợp cổng thanh toán (Visa, Mastercard, ví điện tử) để người dùng có thể thanh toán trực tuyến.
- Thiếu quy trình đặt chỗ thực tế: Form tìm kiếm chỉ hiển thị kết quả mô phỏng, không có quy trình đặt chỗ thực tế với xác nhận, email thông báo, và quản lý đơn hàng.

Về tối ưu hóa và tính năng nâng cao:

- Tìm kiếm còn đơn giản: Hệ thống tìm kiếm hiện tại chỉ là form tìm kiếm cơ bản, chưa có tính năng tìm kiếm nâng cao với nhiều bộ lọc phức tạp, autocomplete, hoặc tìm kiếm theo từ khóa.
- Thiếu tính năng so sánh: Người dùng không thể so sánh nhiều tour hoặc khách sạn cùng lúc.
- Thiếu tính năng yêu thích/wishlist: Người dùng không thể lưu các tour hoặc điểm đến yêu thích để xem lại sau.
- Thiếu đa ngôn ngữ: Website chỉ hỗ trợ tiếng Việt, chưa hỗ trợ các ngôn ngữ khác để phục vụ khách hàng quốc tế.

5.3. Hướng phát triển

5.3.1. Đề xuất cải tiến về backend và cơ sở dữ liệu

Để nâng cấp website từ static website sang dynamic website với đầy đủ chức năng, cần phát triển các thành phần sau:

Xây dựng hệ thống backend:

- Công nghệ đề xuất: Node.js với Express.js hoặc Python với Django/Flask, hoặc PHP với Laravel.
- Chức năng: Xử lý các request từ frontend, quản lý business logic, xác thực người dùng, và tương tác với database.
- API RESTful: Xây dựng API để frontend có thể giao tiếp với backend, cho phép tách biệt frontend và backend, dễ dàng phát triển ứng dụng mobile sau này.

Xây dựng cơ sở dữ liệu:

- Công nghệ đề xuất: MySQL, PostgreSQL.

Tích hợp API bên thứ ba:

- Tích hợp với API của các hãng hàng không để lấy thông tin chuyến bay real-time.
- Tích hợp với API của các hệ thống khách sạn (Booking.com API, Agoda API) để lấy thông tin và giá cả cập nhật.
- Tích hợp với Google Maps API để hiển thị bản đồ và chỉ đường.

5.3.2. Đề xuất cải tiến về chức năng người dùng

Hệ thống đăng nhập và đăng ký:

- Xây dựng trang đăng ký với validation đầy đủ (email, password strength, xác nhận email).
- Xây dựng trang đăng nhập với tính năng "Remember me" và "Forgot password".
- Sử dụng JWT (JSON Web Token) hoặc session để quản lý authentication.
- Tích hợp đăng nhập qua mạng xã hội (Google, Facebook) để tăng trải nghiệm người dùng.

Quản lý tài khoản người dùng:

- Trang profile để người dùng xem và chỉnh sửa thông tin cá nhân.
- Trang lịch sử đặt chỗ để xem các tour, khách sạn đã đặt.
- Tính năng yêu thích/wishlist để lưu các tour và điểm đến yêu thích.
- Thông báo về các ưu đãi và tour mới dựa trên sở thích.

Tính năng đánh giá và bình luận:

- Cho phép người dùng đánh giá và bình luận về tour, khách sạn, điểm đến sau khi sử dụng dịch vụ.
- Hiển thị đánh giá trung bình và số lượng đánh giá.
- Tính năng "Helpful" để đánh giá tính hữu ích của bình luận.

- Moderation system để kiểm duyệt bình luận không phù hợp.

5.3.3. Đề xuất cải tiến về quản trị hệ thống

Hệ thống quản trị (CMS - Content Management System):

- Trang admin với dashboard hiển thị thống kê (số lượng đơn hàng, doanh thu, người dùng).
- Quản lý điểm đến: thêm, sửa, xóa điểm đến với editor rich text.
- Quản lý tour: tạo và chỉnh sửa tour với lịch trình chi tiết, giá cả, hình ảnh.
- Quản lý khách sạn: thêm khách sạn, quản lý phòng, giá cả, tiện ích.
- Quản lý đơn đặt hàng: xem, cập nhật trạng thái, xử lý hủy đơn.
- Quản lý đánh giá: kiểm duyệt, xóa đánh giá không phù hợp.

Hệ thống phân quyền:

- Phân quyền theo vai trò: Super Admin, Admin, Manager, Staff, User.
- Mỗi vai trò có quyền truy cập khác nhau vào các chức năng của hệ thống.
- Audit log để ghi lại các thao tác quan trọng của admin.

5.3.4. Đề xuất cải tiến về tính năng nâng cao

Tìm kiếm nâng cao:

- Tìm kiếm theo từ khóa với autocomplete và gợi ý.
- Bộ lọc phức tạp: giá cả, đánh giá, khoảng cách, tiện ích, thời gian.
- Tìm kiếm theo vị trí địa lý sử dụng GPS hoặc bản đồ.
- Lưu lịch sử tìm kiếm và gợi ý dựa trên lịch sử.

Tính năng so sánh:

- Cho phép người dùng so sánh nhiều tour hoặc khách sạn cùng lúc.
- Hiển thị bảng so sánh với các tiêu chí: giá, thời gian, đánh giá, tiện ích.

Tính năng mạng xã hội:

- Chia sẻ tour và điểm đến lên mạng xã hội (Facebook, Twitter, Instagram).
- Tích hợp Facebook Comments hoặc Disqus cho bình luận.
- Hiện thị số lượt chia sẻ và like.

Chương 5 đã tổng kết toàn bộ quá trình thực hiện đề tài thiết kế và xây dựng website du lịch Travel Explorer. Đề tài đã đạt được các mục tiêu ban đầu, hoàn thành đầy đủ các yêu cầu về thiết kế giao diện, phát triển chức năng cơ bản, responsive design và tối ưu hiệu suất. Mặc dù còn một số hạn chế về backend, database và các tính năng nâng cao do giới hạn về phạm vi đề án, nhưng những gì đã đạt được đã tạo nền tảng vững chắc cho việc phát triển tiếp theo. Các đề xuất hướng phát triển trong tương lai đã được trình bày chi tiết, bao gồm việc xây dựng backend, database, hệ thống thanh toán, CMS, và các tính năng nâng cao khác. Những đề xuất này không chỉ giúp hoàn thiện sản phẩm mà còn mở ra nhiều hướng nghiên cứu và phát triển tiếp theo, góp phần nâng cao giá trị ứng dụng thực tế của website trong tương lai.

TÀI LIỆU THAM KHẢO

- [1]. <https://www.w3schools.com>, “Khái niệm về HTML, CSS, JavaScript, truy cập ngày 8 tháng 12 năm 2025”
- [2]. <https://www.geeksforgeeks.org>, “Khái niệm về HTML, truy cập ngày 15 tháng 11 năm 2025”
- [3]. <https://fullstack.edu.vn/landing/htmlcss>, “Cách dễ nhất để học HTML/ CSS cho người mới bắt đầu, truy cập ngày 10 tháng 11 năm 2025”
- [4]. <https://www.envato.com/products/>, “Mẫu website, đồ họa, hình ảnh, truy cập ngày 2 tháng 12 năm 2025”
- [5]. <https://www.traveloka.com/vi-vn>, “Giao diện và chức năng của website du lịch trực tuyến, truy cập ngày 10 tháng 12 năm 2025”
- [6]. <https://www.booking.com/>, “Giao diện và chức năng của website du lịch trực tuyến, truy cập ngày 10 tháng 12 năm 2025”
- [7]. <https://travel.com.vn/>, “Giao diện và chức năng của website du lịch trực tuyến, truy cập ngày 10 tháng 12 năm 2025”
- [8]. <https://web.dev/fast/>, “Cách thực hành tốt nhất về hiệu suất cho nhà phát triển web, truy cập ngày 15 tháng 11 năm 2025”
- [9]. <https://www.w3.org/WAI/fundamentals/accessibility-intro/>, “Giới thiệu về Khả năng Truy cập Web (Web Accessibility), truy cập ngày 20 tháng 11 năm 2025”
- [10]. <https://www.google.com/search?q=https://www.smashingmagazine.com/2018/01/html-semantics/>, “Tầm Quan Trọng của HTML Ngữ Nghĩa (Semantic HTML), truy cập ngày 16 tháng 11 năm 2025”
- [11]. <https://www.google.com/search?q=https://www.sitepoint.com/beginners-guide-javascript-dom/>, “Hướng Dẫn Cơ Bản về DOM trong JavaScript, truy cập ngày 7 tháng 12 năm 2025”
- [12]. <https://developers.google.com/speed/pagespeed/insights/>, “Công cụ Phân tích và Tối ưu hóa Tốc độ Tải Trang, truy cập ngày 8 tháng 12 năm 2025”

PHỤ LỤC

[1]. Link Github:

https://github.com/phamquockhanh0209/thiet_ke_web_topic_travel